A PROJECT REPORT ON

"CHAINSHIELD TRANSPORTATION SYSTEM"

Real-Time Disruption Prediction & Route Optimisation for Logistics

Submitted in Fulfillment of the Requirement

for Completion of Semester – 3 in

Master of Computer Application (MCA)

Academic Year 2026-27

To

LJ INSTITUTE OF COMPUTER APPLICATION

Affiliated to LJK University

Guided By:
Prof. Dhaval Shah

Prepared By: (Group No. 59)
Bhavik A. Visani (D51 – 25004401110170)
Diya Shah (D36 – 25003401110017)
Shruti Somani (D42 – 25004401110259)

DECLARATION

We, the undersigned students of Group No. 59, Master of Computer Application (MCA) Semester 3 at LJ Institute of Computer Application (affiliated to LJK University), hereby declare that the project entitled “ChainShield Transportation System” is our own original work, carried out under the guidance of Prof. Dhaval Shah, and has not been submitted, in whole or in part, for the award of any other degree, diploma, or certificate to this or any other institute or university.

We further declare that all the information, technical concepts, and design decisions presented in this report have been gathered, analysed, and implemented by us to the best of our understanding, and that any reference material used has been duly acknowledged in the Bibliography.

Date: _____ Place: Ahmedabad

Bhavik A. Visani (D51) _____

Diya Shah (D36) _____

Shruti Somani (D42) _____

CERTIFICATE

This is to certify that the project entitled “ChainShield Transportation System”, submitted for MCA Semester 3, has been carried out by Bhavik A. Visani (D51), Diya Shah (D36), and Shruti Somani (D42) of Group No. 59, under my guidance and supervision.

The work presented in this report is a bona fide record of the project carried out by the above-named students and, to the best of my knowledge, has not been submitted elsewhere for the award of any degree or diploma.

I wish them success in their future endeavours.

Prof. Dhaval Shah	**Head of Department / Director**
Project Guide	LJ Institute of Computer Application

PREFACE

Global trade today moves through supply chains of extraordinary scale and complexity — millions of shipments crossing ports, highways, railways, and air corridors every single day. The same forces that make this scale possible also make it fragile: a single storm, a congested port, or a mishandled hazardous cargo shipment can ripple into delays that cost businesses time, money, and customer trust. Historically, these disruptions have been discovered only after the fact, when a delivery has already slipped past its promised window.

This project, ChainShield Transportation System, was conceived to address exactly that gap. Rather than treating disruption management as a reactive, after-the-event exercise, ChainShield continuously evaluates every active shipment against a weighted risk model, flags the ones heading toward trouble, and proposes concrete rerouting alternatives before the delay materialises.

The idea came from a practical observation: most logistics teams still learn about a delay only after it has happened. We set out to build ChainShield as a lean, self-hostable product — Python (Flask), SQLite, server-rendered Jinja2/Bootstrap templates and Docker — that any logistics company can run on its own server without paid services or a complex cloud setup, while still delivering real-time risk scoring, route recommendations and alerts.

This report documents the problem definition, requirement analysis, system design, implementation, and testing of the ChainShield Transportation System, prepared as part of the curriculum prescribed for MCA Semester 3 at LJ Institute of Computer Application.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported us through the course of this project.

First and foremost, we thank LJK University and LJ Institute of Computer Application for providing us the opportunity and the academic environment to undertake this project as part of our MCA Semester 3 curriculum.

We are deeply grateful to our project guide, Prof. Dhaval Shah, for his valuable guidance, constructive feedback, and constant encouragement throughout the planning, design, and documentation of this project. His inputs have been instrumental in shaping the direction of ChainShield Transportation System.

We also extend our thanks to the Director and faculty of LJ Institute of Computer Application for their support, and to our classmates and peers who offered helpful suggestions during discussions on this project.

Finally, we thank our families for their patience and encouragement throughout this endeavour.

Thank you,

Bhavik A. Visani, Diya Shah, Shruti Somani

Group No. 59, MCA Semester 3

INDEX

Sr. No.	Particulars	Page No.
1	**Introduction**	7
	1.1 Problem Definition	7
	1.2 Objective of the New System	7
	1.3 Core Components	8
	1.4 Project Profile	9
	1.5 Need of the New System	9
	1.6 Scope of the New System	10
	1.7 Advantages and Limitations of the Proposed System	11
	1.8 Proposed Time Line Chart	11
	1.9 Targeted Users	13
2	**Requirement Determination and Analysis**	15
	2.1 Requirement Determination	15
	2.2 Requirement Specification	15
3	**System Design**	18
	3.1 Use Case Diagram	18
	3.2 Data Dictionary	21
	3.3 User Interface Design	28
	3.4 Report Design	32
4	**Development**	35
	4.1 Coding Standards	35
	4.2 Implementation Details	39
	4.3 Output Screens	40
5	**Testing**	49
	5.1 Test Cases	49
6	**Proposed Enhancement**	53
7	**Conclusion**	54
8	**Bibliography**	55

1. INTRODUCTION

1.1 Problem Definition

Logistics companies move hundreds of shipments at the same time across road, rail, sea and air. Every one of these shipments is exposed to risks that are outside the company's control — heavy rain and storms, congestion at hubs and ports, customs holds, vehicle breakdowns and the special handling needs of sensitive cargo such as perishable or hazardous goods.

In most companies these disruptions are noticed only after a delivery date has already slipped. Managers depend on manual tracking — phone calls to drivers, spreadsheets and e-mail — and learn about a problem only when a customer or a downstream partner reports a missed delivery. By that time the shipment is already late, the alternate routes that were available earlier have narrowed, and the cost of recovery (express freight, penalty clauses and lost customer trust) has multiplied.

The core problem, therefore, is the absence of a continuously running system that scores the disruption risk of every active shipment, warns the right people before the delay actually happens, and recommends a corrective route with its time, cost and risk trade-offs clearly stated. ChainShield Transportation System is designed to fill exactly this gap.

1.2 Objective of the New System

The main objective of ChainShield is to move logistics operations from a reactive way of working to a preemptive one. The specific objectives of the new system are described below.

- **Continuous risk analysis** — to analyse weather, route congestion, cargo sensitivity and historic delay data for every active shipment on a fixed schedule, without anyone having to start the check manually.
- **Early disruption detection** — to detect and flag a potential disruption before the delivery timeline is compromised, using a transparent, weighted risk-scoring formula.
- **Optimised route recommendations** — to automatically generate ranked alternatives (hold at a hub, reroute through another hub, or switch the transport mode) the moment a shipment crosses the risk threshold.
- **Instant communication** — to notify managers and the assigned field operator in real time when a disruption is detected, escalated or resolved, closing the gap between decision and execution.

- **Role-based working** — to give administrators, managers and field operators (drivers, loco pilots, pilots and captains) their own dashboards containing only the information relevant to their work.
- **Low-cost deployment** — to keep the system self-contained and inexpensive, built on an open-source Python/Flask stack and packaged with Docker so that any logistics company can run it on its own server.

1.3 Core Components

ChainShield is made up of seven major components. Each component is implemented as a separate module so that it can be developed and tested independently.

- **Risk Engine** — a scheduled background job that scores every active shipment every 60 seconds using a four-factor weighted model (weather severity, route congestion, cargo sensitivity and historic delay probability) and classifies it as LOW, AT-RISK, MEDIUM or HIGH.
- **Disruption Manager** — converts a MEDIUM-or-higher risk score into a formal disruption record, escalates it to HIGH if the risk keeps rising, and asks the route solver for alternatives.
- **Route Solver** — builds hold, hub-reroute and mode-switch alternatives using great-circle distances between hubs, and ranks them by a weighted score of residual risk, extra time and extra cost in rupees.
- **Notification and Chat Module** — pushes real-time alerts over Flask-SocketIO to admins, managers and the assigned field operator, and lets users send direct messages to each other.
- **Admin Panel and Reports** — manages users, logistics hubs and cargo types, keeps an activity audit log, and produces reports with date-range filters and CSV export.
- **AI Assistant** — a conversational assistant based on the Google Gemini API, with an offline fallback engine, that answers natural-language questions using live shipment and disruption data.
- **Live Map** — a Leaflet.js map of shipment positions, planned routes, hubs and shaded disruption zones that refreshes automatically after every risk scan.

1.4 Project Profile

The project profile below gives a one-page summary of the project, the team that developed it and the technologies used.

Table 1.1 — Project Profile

Particulars	Details
Project Title	ChainShield Transportation System
Project Group	Group No. 59
Team Members	Bhavik A. Visani (Roll No. D51, Enrollment No. 25004401110170) Diya Shah (Roll No. D36, Enrollment No. 25003401110017) Shruti Somani (Roll No. D42, Enrollment No. 25004401110259)
Project Guide	Prof. Dhaval Shah
Domain	Logistics & Supply Chain Management — real-time disruption prediction and route optimisation
Project Description	A web-based system that continuously scores active shipments for disruption risk, opens disruptions automatically, recommends ranked route alternatives and notifies the responsible people in real time, with role-based dashboards for admins, managers and field operators.
Front End	HTML5, CSS3, Bootstrap 5, Jinja2 templates, JavaScript, Leaflet.js, Chart.js
Back End	Python 3.10, Flask, Flask-SQLAlchemy
Database	SQLite
Other Tools & APIs	Flask-Login, Flask-SocketIO, Flask-WTF, APScheduler, Google Gemini API, OpenWeatherMap API, Docker, Git / GitHub
Platform	Windows 11 / any operating system that supports Docker
Duration	August 2026 – November 2026 (Semester 3)
Submitted To	LJ Institute of Computer Application (LJK University)
Academic Year	2026-27

1.5 Need of the New System

A study of how logistics teams currently track their shipments showed several weaknesses in the existing, largely manual process. These weaknesses create the need for the new system.

- **Reactive monitoring** — disruptions surface only after a customer complaint or a missed delivery commitment, when recovery options are already limited and expensive.
- **Tracking without prediction** — existing tracking screens show where a shipment is, but not what is about to go wrong with it. There is a clear gap between tracking and prediction.
- **Signals in silos** — weather, congestion, cargo type and past delay information sit in different places, and no single process combines them into one actionable risk score per shipment.

- **Ad hoc rerouting** — rerouting decisions depend on a manager's personal experience instead of a systematic comparison of options with their time, cost and risk clearly quantified.
- **Slow communication** — field operators usually learn about a rerouting decision late, over a phone call or radio, which adds further delay and the chance of miscommunication.
- **No audit trail** — there is no reliable record of who took which decision and when, which makes review and accountability difficult.

1.6 Scope of the New System

The scope defines what the current version of ChainShield covers and what is intentionally left for future versions.

Included in the scope

- Interval-based (near-real-time) risk scoring of every active shipment.
- Automatic creation of a disruption when a shipment crosses the risk threshold, with ranked reroute and hold solutions.
- A manager dashboard for reviewing and resolving disruptions, and a field-operator dashboard for status updates and resolution notifications.
- An admin panel for managing users, hubs and cargo types, along with an activity audit log.
- Shipment management (create, view, edit and delete) with server-side validation.
- Reports with date-range filters and CSV export — disruption summary, resolution time, shipment risk and audit log.
- An AI assistant for natural-language questions about shipment data, and a live map of shipments and disruption zones.
- A demo mode — simulated shipment movement, a one-click “Trigger demo disruption” action and a reset command — so the system can be demonstrated without live production data.

Outside the scope of the current version

- Integration with live carrier GPS or telematics hardware (simulated positions are used instead).
- Payment, billing and contractual SLA-enforcement modules.

- Multi-tenant support for several independent companies on one installation.

1.7 Advantages and Limitations of the Proposed System

Advantages

- **Preemptive handling** — the organisation reacts to a risk before it becomes a delay, instead of after the delivery has been missed.
- **Transparent scoring** — risk is measured with a clear, weighted formula that anyone can verify, instead of by gut feeling.
- **Faster response** — the time between “risk detected” and “operator notified” is reduced to a few seconds.
- **Less information overload** — role-based views show each user only what is relevant to their job.
- **Cheap and portable** — the lightweight Flask and SQLite stack runs inside a single Docker container, so it is inexpensive to host and easy to set up on any machine.

Limitations

- **Near-real-time, not instantaneous** — freshness depends on the scheduler interval, which is 60 seconds by default.
- **Data quality** — weather and congestion signals come from public APIs or a simulated fallback rather than certified logistics-grade data feeds.
- **Single-server database** — SQLite suits one server; a large multi-branch deployment would need PostgreSQL or MySQL.
- **AI answers are advisory** — the assistant is only as reliable as the Gemini model and the data given to it; it is a convenience feature, not a system of record.

1.8 Proposed Time Line Chart

The project was planned phase-wise across the semester, from August to November 2026. In the chart below each month is divided into four weeks (W1 – W4) and the shaded cells show the weeks in which each phase is carried out. Documentation runs in parallel with every other phase, from requirement gathering until final submission.

Fig 1.1 — Proposed Time Line Chart (Aug – Nov 2026)

Sr.	Phase	Aug 2026				Sep 2026				Oct 2026				Nov 2026			
		W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4
1	Requirement Gathering (R.G.)		■														
2	Requirement Analysis & SRS		■	■													
3	System Design (Use Case, DB, UI)			■	■												
4	Environment & Docker Setup			■													
5	Coding / Development			■	■	■	■	■									
6	UI Enhancement (Themes, Mobile)							■	■								
7	Testing							■	■	■							
8	Implementation / Deployment									■	■						
9	Documentation		■	■	■	■	■	■	■	■	■	■	■	■			
10	Final Review, Submission & Viva												■	■	■		

Note: W1 = days 1–7, W2 = 8–14, W3 = 15–21, W4 = 22 to month-end. Shaded cells show the weeks in which each phase is carried out.

Table 1.2 records the milestones actually achieved against this plan, taken from the project's version-control history and document records.

Table 1.2 — Actual Progress Against the Plan

Milestone	Completed On	Status
Requirement gathering, problem analysis and technology stack decision	13 Aug 2026	Done
Requirement analysis and system design in the project report	15 – 19 Aug 2026	Done
Development environment and Docker setup	16 Aug 2026	Done
Authentication and base user interface	16 – 25 Aug 2026	Done
SRS document and first presentation	25 Aug 2026	Done
Risk engine scan loop, route solver, disruption module, admin panel	7 – 8 Sep 2026	Done
Shipment CRUD, live map, dashboards, charts, real-time notifications	9 – 16 Sep 2026	Done
Reports with CSV export, cargo/hub/user management, demo mode	17 – 20 Sep 2026	Done
Testing (101 automated checks) and Docker deployment verification	21 – 22 Sep 2026	Done
UI enhancement — responsive layout, light/dark themes, animations	22 – 24 Sep 2026	Done
Final documentation revised as per guide's review	30 Sep 2026	Done
Final submission and viva	As per university schedule	Pending

1.9 Targeted Users

ChainShield is designed for the people who plan, supervise and carry out the movement of goods inside a logistics company. Each type of user has a different responsibility, and the system gives each of them a separate, role-based view. The targeted users are described in detail below.

1. System Administrator

The administrator is the person responsible for setting up and maintaining the system for the whole company — usually a member of the IT or operations-support team. The administrator creates user accounts and assigns their roles, activates or deactivates accounts, maintains the master data of logistics hubs (base cities) and cargo types along with their sensitivity weights, and reviews the system configuration such as the risk weights and API integrations. The administrator also has access to every report and to the complete activity audit log, which records logins, administrative changes and disruption decisions. Because the administrator also inherits all the rights of a manager, he or she can step in to resolve a disruption when needed. Only an existing administrator can create another administrator — admin rights can never be self-assigned at registration.

2. Logistics / Supply Chain Manager

The manager is the primary decision-maker and the main user of ChainShield. Managers create and update shipments, assign each shipment to a field operator, and watch the dashboard, the risk watch-list and the live map to see which shipments are heading towards trouble. When the risk engine opens a disruption, the manager receives an instant alert, compares the ranked route options (with their extra time, extra cost and residual risk) and applies the most suitable one with a single click. Managers can also ask the AI assistant questions such as “Which shipments are at high risk?”, view and export reports for a chosen date range, trigger a demo disruption during presentations, and chat directly with field operators. For the manager, the system replaces phone calls and spreadsheets with a single, data-driven control screen.

3. Field Operators — Driver, Loco Pilot, Pilot and Captain

Field operators are the people who physically move the cargo: truck drivers for road shipments, loco pilots for rail, pilots for air cargo and ship captains for sea freight. They are usually on the move and need a simple, mobile-friendly view. After logging in, a field operator sees only the shipments assigned to him or her, their current risk level and position on the live map. Operators update the status of a shipment (In-transit, Delayed, On-hold or

Delivered), and the moment a manager resolves a disruption on their shipment they receive a real-time notification describing the new route or the hold instruction. They can also chat directly with their manager or the admin to clarify instructions. This removes the delay and confusion of passing instructions over the phone or radio.

4. Background Scheduler (System Actor)

Although it is not a human user, the background scheduler is an important actor of the system. It wakes up every 60 seconds, recalculates the risk score of every in-transit shipment, opens or escalates disruptions, asks the route solver for alternatives and triggers the notifications — all without any human action.

5. Indirect Beneficiaries

End customers, consignees and downstream business partners do not log in to ChainShield, but they benefit directly from fewer missed deliveries and more reliable delivery estimates. Company management also benefits from the reports on disruption frequency, resolution time and value at risk, which support planning and performance review.

Table 1.3 — Targeted Users and Their Access

User	Main Responsibilities	Access in ChainShield
Administrator	System set-up, users, master data, audit	Full access — admin panel, all reports, audit log, plus all manager functions
Manager	Shipment planning, disruption decisions	Shipments CRUD, disruptions, route decisions, reports, AI assistant, chat, live map
Field Operator (Driver / Loco Pilot / Pilot / Captain)	Moving the cargo, status updates	Own assigned shipments only, status update, notifications, chat, live map
Scheduler (system)	Automatic risk scanning	Risk engine, disruption creation, route generation, alerts

2. REQUIREMENT DETERMINATION AND ANALYSIS

2.1 Requirement Determination

The requirements of ChainShield Transportation System were determined through group discussion, a study of how logistics operators track shipments today (manual follow-ups, spreadsheets and phone calls), and an analysis of the roles involved — administrators, operations managers and field operators (drivers, loco pilots, pilots and ship captains).

The group divided the problem into three questions: what a supply-chain manager needs to see, what a field operator needs to act on, and what an administrator needs to configure. Each requirement was then checked against the main objective of catching disruptions before they cascade, rather than after. The key requirements identified through this process are:

1. All users (admin, manager and field operator) must log in before they can access the system.
2. The system must continuously evaluate the risk of every active shipment without anyone triggering it manually.
3. A disruption must be created automatically when a shipment's risk score reaches MEDIUM (≥ 50), and escalated when it rises to HIGH (≥ 70).
4. The system must propose at least three ranked route alternatives for every disruption (hold, reroute through a hub, mode switch or continue).
5. Field operators must be notified in the application the moment a manager resolves a disruption on their shipment.
6. Administrators must be able to manage users, hubs and cargo types, and view an audit trail of system activity.
7. The system must be demonstrable without live production data, through a simulated demo-disruption trigger.

2.2 Requirement Specification

Functional Requirements

Functional requirements describe what the system must do:

1. User registration, login and role-based access control (Admin, Manager, Field Operator).

2. Create, view, update and delete shipments (managers and admins); field operators see only their assigned shipments.
3. Continuous, scheduled risk scoring of all active shipments.
4. Automatic disruption detection and alerting once a risk threshold is crossed.
5. Generation of ranked route alternatives (hold, hub reroute, mode switch, continue) with cost, time and residual-risk trade-offs.
6. Real-time in-app notifications (Socket.IO) to managers and the assigned field operator on detection, escalation and resolution of a disruption.
7. Direct messaging (chat) between users.
8. Admin CRUD operations for users, logistics hubs and cargo types.
9. Activity audit log of all administrative and disruption-resolution actions.
10. AI assistant for natural-language questions about the supply chain.
11. Reports with a date-range filter and CSV export.
12. Live map of shipment positions and disruption zones.
13. A one-click demo-mode trigger for presentations.

Non-Functional Requirements

Non-functional requirements describe how well the system must perform:

- **Performance** — risk scores must be refreshed at least once every 60 seconds.
- **Usability** — each role sees a dashboard tailored to its work; the interface is responsive from phone to desktop, offers light and dark themes and respects the reduced-motion accessibility setting.
- **Portability** — the system must run identically on any machine through Docker, without manual environment setup.
- **Reliability** — database writes must be transactional, and the background scheduler must recover gracefully from a failed run.
- **Security** — passwords are stored only as hashes; sessions are managed by Flask-Login; all forms are CSRF-protected; admin rights cannot be self-assigned at registration.
- **Maintainability** — code is organised into modular Flask blueprints so features can be extended independently.

Hardware and Software Requirements

Table 2.1 — Hardware and Software Requirements

Component	Specification
Processor / RAM	Intel i3 or equivalent / minimum 4 GB RAM (8 GB recommended)
Operating System	Windows 11 / any operating system supporting Docker
Backend Language	Python 3.10
Web Framework	Flask, Flask-SQLAlchemy, Flask-Login, Flask-SocketIO, Flask-WTF, APScheduler
Database	SQLite
Containerisation	Docker, Docker Compose (Gunicorn + eventlet worker)
Front-End Tools	HTML5, CSS3, Bootstrap 5, Bootstrap Icons, Jinja2, JavaScript, Leaflet.js, Chart.js
External APIs	Google Gemini API, OpenWeatherMap API
Browser	Google Chrome / Microsoft Edge / Mozilla Firefox (latest)
Documentation Tool	Microsoft Word 2021, Microsoft PowerPoint 2021

3. SYSTEM DESIGN

3.1 Use Case Diagram

The use case diagram (Fig 3.1) shows what each actor can do inside the boundary of ChainShield Transportation System. It has three human actors — Admin, Manager and Field Operator — and one system actor, the Scheduler. Every association is drawn as a straight line and no two lines overlap.

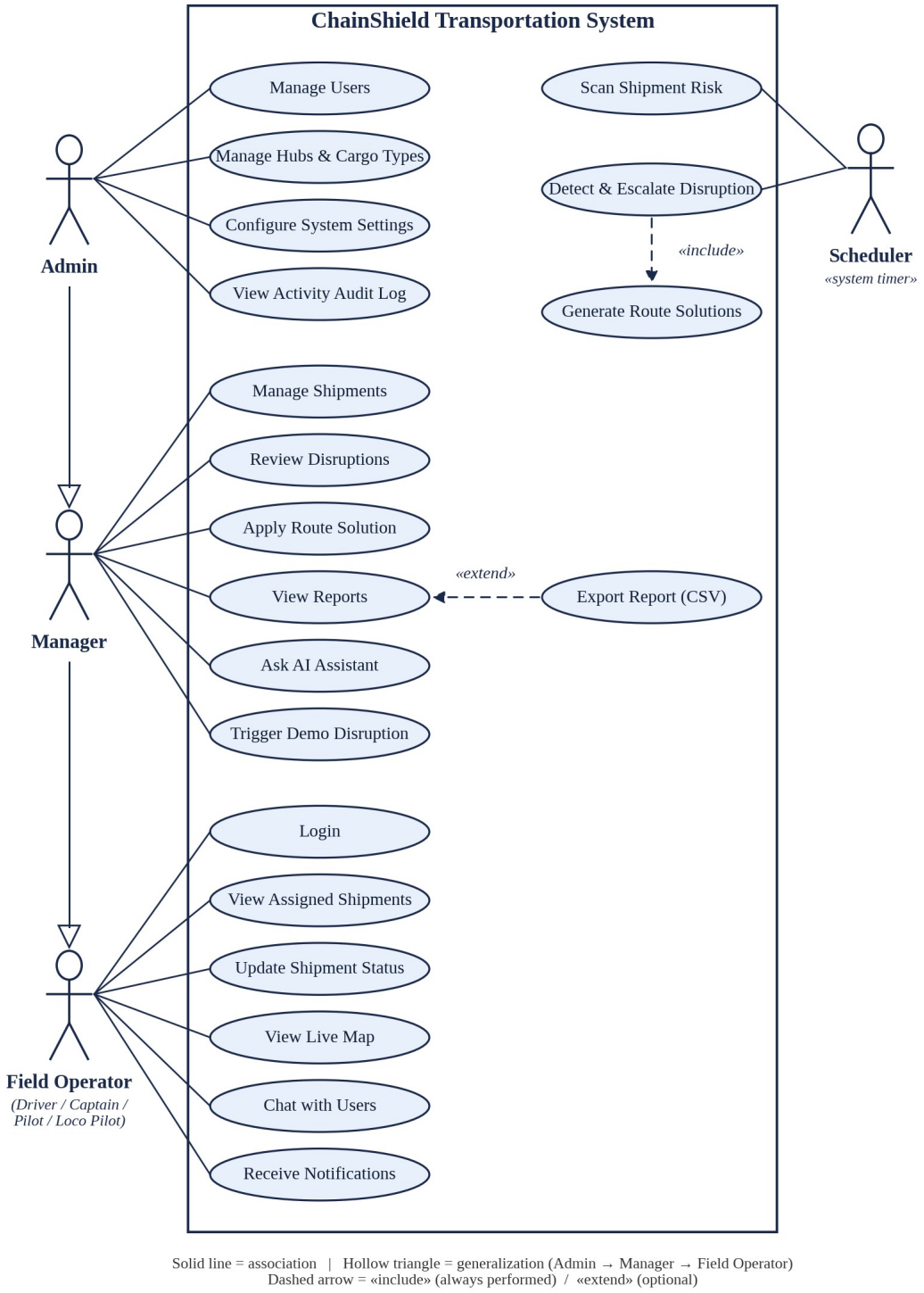


Fig 3.1 — Use Case Diagram

Actors and generalization

The three human actors are connected by generalization (the hollow triangle). A Manager is a specialised Field Operator, and an Admin is a specialised Manager. This means the Manager automatically inherits all Field Operator use cases (Login, View Assigned Shipments, Update Shipment Status, View Live Map, Chat with Users and Receive Notifications), and the

Admin inherits everything a Manager can do. Because of this, each actor is connected only to the use cases that are new for that role, which keeps the diagram free of crossing lines.

Example of «include»

“Detect & Escalate Disruption” «include» “Generate Route Solutions”. An include relationship means the included use case is always performed as a mandatory part of the base use case. Whenever the Scheduler detects or escalates a disruption, the route solver is always run to generate the ranked hold, reroute and mode-switch options — a disruption is never created without them.

Example of «extend»

“Export Report (CSV)” «extend» “View Reports”. An extend relationship means the extending use case is optional and happens only under a certain condition. A manager normally just views a report on screen; only if he or she clicks the Export CSV button is the report additionally downloaded as a CSV file. The arrow points from the optional behaviour to the base use case.

System Architecture

The system follows a layered architecture (Fig 3.2): a Flask + Jinja2 + Bootstrap presentation layer (with Leaflet.js and Chart.js) talks to the server over HTTP and WebSockets; core services (Risk Engine, Route Solver, Notifications/Chat, AI Assistant, Admin and Reports) form the middle layer; a Flask-SQLAlchemy data-access layer sits below them; and a SQLite database forms the base. The entire stack is packaged inside one Docker container.

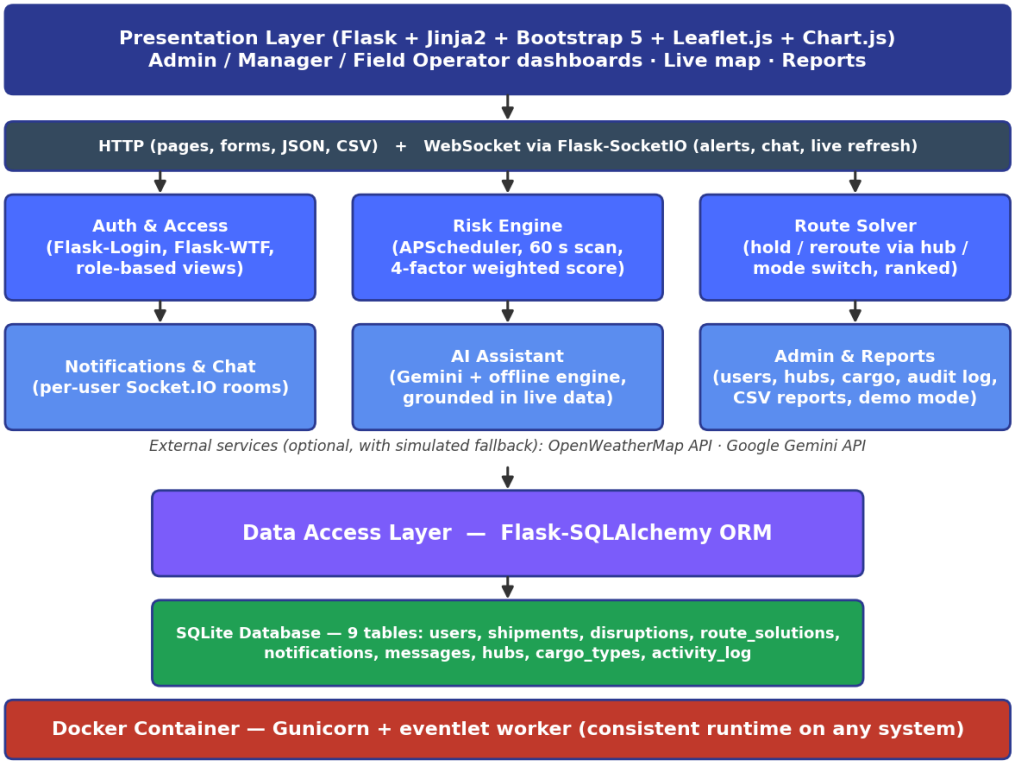


Fig 3.2 — ChainShield System Architecture

3.2 Data Dictionary

The database of ChainShield is named chainshield and is stored in the SQLite file instance/chainshield.db. It contains nine tables, one for each model class in app/models.py. For every table, the data dictionary below gives its name, description, primary key and foreign keys, followed by each field with its data type, size, constraint and description. The last column shows the sample data of one record (id = 1) taken from the demo database.

Conventions used in the tables: every primary key is an INTEGER that SQLite generates automatically; NULL means the field is optional; DEFAULT values are filled in by the application when a new record is saved; a size is given only for VARCHAR fields, and "—" means the type has no size; all dates and times are stored in UTC.

1. Table: users

Description: Stores the login account, role and status of every person who uses the system.

Primary Key: id **Foreign Key(s):** None

Table 3.1 — users

Sr. No.	Field Name	Data Type	Size	Constraint	Description	Sample Data (id = 1)
1	id	INTEGER	—	PRIMARY KEY	Unique user identifier, generated automatically	1
2	name	VARCHAR	100	NOT NULL	Full name of the user	Admin
3	email	VARCHAR	100	NOT NULL, UNIQUE	Login e-mail address (indexed for fast login)	admin@chainshield.com
4	password_hash	VARCHAR	255	NOT NULL	Werkzeug (scrypt) hash of the password; the plain password is never stored	scrypt:32768:8:1$jCqB…
5	role	VARCHAR	20	NOT NULL, DEFAULT 'manager'	admin / manager / captain / pilot / driver / loco_pilot	admin
6	phone	VARCHAR	20	NULL	Contact number (optional)	NULL
7	is_active	BOOLEAN	—	NOT NULL, DEFAULT 1	1 = active; 0 = deactivated by an admin and cannot log in	1
8	created_at	DATETIME	—	DEFAULT current date-time (UTC)	Date and time the account was created	2026-08-25 03:48:26

2. Table: hubs

Description: Master data of logistics hubs (base cities) used for plotting, rerouting and holding shipments.

Primary Key: id **Foreign Key(s):** None

Table 3.2 — hubs

Sr. No.	Field Name	Data Type	Size	Constraint	Description	Sample Data (id = 1)
1	id	INTEGER	—	PRIMARY KEY	Unique hub identifier, generated automatically	1
2	name	VARCHAR	100	NOT NULL	Name of the hub	Ahmedabad Hub
3	location	VARCHAR	150	NULL	City and state of the hub	Ahmedabad, Gujarat
4	lat	FLOAT	—	NULL	Latitude, used for the map marker and detour distance	23.0225
5	lng	FLOAT	—	NULL	Longitude, used for the map marker and detour distance	72.5714

3. Table: cargo_types

Description: Master data of cargo categories and how sensitive each one is to disruption.

Primary Key: id **Foreign Key(s):** None

Table 3.3 — cargo_types

Sr. No.	Field Name	Data Type	Size	Constraint	Description	Sample Data (id = 1)
1	id	INTEGER	—	PRIMARY KEY	Unique cargo type identifier, generated automatically	1
2	name	VARCHAR	50	NOT NULL, UNIQUE	Cargo category name	General
3	sensitivity_weight	FLOAT	—	DEFAULT 0.5, range 0.0 – 1.0	Sensitivity of the cargo; × 100 gives the cargo factor of the risk score	0.3

4. Table: shipments

Description: Every shipment with its route, cargo, assigned operator, live position and current risk.

Primary Key: id **Foreign Key(s):** cargo_type_id → cargo_types(id), assigned_operator_id → users(id), hub_id → hubs(id)

Table 3.4 — shipments

Sr. No.	Field Name	Data Type	Size	Constraint	Description	Sample Data (id = 1)
1	id	INTEGER	—	PRIMARY KEY	Unique shipment identifier, generated automatically	1
2	tracking_no	VARCHAR	30	NOT NULL, UNIQUE	Tracking number (auto-generated as CS-xxxx if left blank)	CS-1001
3	origin	VARCHAR	100	NULL	Origin city	Ahmedabad
4	destination	VARCHAR	100	NULL	Destination city	Mumbai
5	cargo_type_id	INTEGER	—	FOREIGN KEY → cargo_types(id), NULL	Type of cargo carried	2 (Perishable)
6	transport_mode	VARCHAR	20	NULL	Road / Rail / Sea / Air	Road
7	status	VARCHAR	20	DEFAULT 'In-transit'	In-transit / Delayed / On-hold / Delivered	In-transit
8	current_lat	FLOAT	—	NULL	Current latitude (simulated movement)	22.30
9	current_lng	FLOAT	—	NULL	Current longitude (simulated movement)	72.62
10	risk_score	FLOAT	—	DEFAULT 0.0, range 0 – 100	Composite risk score from the last scan	0.0
11	risk_level	VARCHAR	15	DEFAULT 'LOW'	LOW / AT-RISK / MEDIUM / HIGH	LOW
12	cargo_value	FLOAT	—	DEFAULT 0.0	Declared cargo value in ₹ (used for value at risk)	42000.00
13	assigned_operator_id	INTEGER	—	FOREIGN KEY → users(id), NULL	Field operator responsible for the shipment	8 (Suresh Yadav, driver)
14	hub_id	INTEGER	—	FOREIGN KEY → hubs(id), NULL	Origin hub of the shipment	1 (Ahmedabad Hub)
15	created_at	DATETIME	—	DEFAULT current date-time (UTC)	Date and time the shipment was created	2026-09-07 06:24:35

5. Table: disruptions

Description: Disruptions opened automatically by the risk engine when a shipment reaches MEDIUM risk.

Primary Key: id **Foreign Key(s):** shipment_id → shipments(id), resolved_by → users(id)

Table 3.5 — disruptions

Sr. No.	Field Name	Data Type	Size	Constraint	Description	Sample Data (id = 1)
1	id	INTEGER	—	PRIMARY KEY	Unique disruption identifier, generated automatically	1
2	shipment_id	INTEGER	—	NOT NULL, FOREIGN KEY → shipments(id)	Affected shipment	3 (CS-1003)
3	type	VARCHAR	30	NULL	Dominant factor: Weather / Congestion / Cargo / History	Cargo
4	severity	VARCHAR	15	NULL	MEDIUM or HIGH (escalated as the risk rises)	HIGH
5	risk_score	FLOAT	—	NULL	Risk score at detection or at the last escalation	79.69
6	status	VARCHAR	15	DEFAULT 'Open'	Open / Resolved	Resolved
7	detected_at	DATETIME	—	DEFAULT current date-time (UTC)	Date and time the disruption was detected	2026-10-10 20:06:57
8	resolved_at	DATETIME	—	NULL	Date and time a route decision was applied	2026-10-11 05:45:02
9	resolved_by	INTEGER	—	FOREIGN KEY → users(id), NULL	Manager or admin who resolved it	1 (Admin)

6. Table: route_solutions

Description: Ranked route options generated by the route solver for each disruption.

Primary Key: id **Foreign Key(s):** disruption_id → disruptions(id)

Table 3.6 — route_solutions

Sr. No.	Field Name	Data Type	Size	Constraint	Description	Sample Data (id = 1)
1	id	INTEGER	—	PRIMARY KEY	Unique solution identifier, generated automatically	1
2	disruption_id	INTEGER	—	NOT NULL, FOREIGN KEY → disruptions(id), deleted with its disruption	Parent disruption	1
3	option_no	INTEGER	—	NOT NULL	Rank of the option; 1 = recommended	1
4	description	TEXT	—	NULL	Readable description of the option	Reroute via Jaipur Hub (+0 km detour, avoids affected corridor)
5	extra_time_hours	FLOAT	—	NULL	Additional transit time in hours	1.5
6	extra_cost	FLOAT	—	NULL	Additional cost in ₹	1501.22
7	risk_score	FLOAT	—	NULL	Residual risk if this option is chosen	23.91
8	selected	BOOLEAN	—	DEFAULT 0	1 = option applied by the manager	0

7. Table: notifications

Description: In-app alerts delivered to individual users in real time.

Primary Key: id **Foreign Key(s):** user_id → users(id)

Table 3.7 — notifications

Sr. No.	Field Name	Data Type	Size	Constraint	Description	Sample Data (id = 1)
1	id	INTEGER	—	PRIMARY KEY	Unique notification identifier, generated automatically	1
2	user_id	INTEGER	—	NOT NULL, FOREIGN KEY → users(id)	Recipient of the alert	1 (Admin)
3	message	VARCHAR	255	NOT NULL	Alert text	MEDIUM risk on CS-1003 (Delhi → Ahmedabad) — Cargo, score 56.53. Route options ready.
4	is_read	BOOLEAN	—	DEFAULT 0	1 = marked as read	0
5	created_at	DATETIME	—	DEFAULT current date-time (UTC)	Date and time the alert was created	2026-10-10 20:06:57

8. Table: messages

Description: Direct chat messages exchanged between two users.

Primary Key: id **Foreign Key(s):** sender_id → users(id), receiver_id → users(id)

Table 3.8 — messages

Sr. No.	Field Name	Data Type	Size	Constraint	Description	Sample Data (id = 1)
1	id	INTEGER	—	PRIMARY KEY	Unique message identifier, generated automatically	1
2	sender_id	INTEGER	—	NOT NULL, FOREIGN KEY → users(id)	User who sent the message	8 (Suresh Yadav)
3	receiver_id	INTEGER	—	NOT NULL, FOREIGN KEY → users(id)	User who receives the message	1 (Admin)
4	chat_id	VARCHAR	10	NULL	Conversation key in the form 'lowerId-higherId'	1-8
5	content	TEXT	—	NOT NULL	Message text	Hello
6	sent_at	DATETIME	—	DEFAULT current date-time (UTC)	Date and time the message was sent	2026-09-24 13:22:49

9. Table: activity_log

Description: Audit trail of logins, administrative actions and disruption decisions.

Primary Key: id **Foreign Key(s):** user_id → users(id)

Table 3.9 — activity_log

Sr. No.	Field Name	Data Type	Size	Constraint	Description	Sample Data (id = 1)
1	id	INTEGER	—	PRIMARY KEY	Unique log entry identifier, generated automatically	1
2	user_id	INTEGER	—	FOREIGN KEY → users(id), NULL	User who performed the action	1 (Admin)
3	action	VARCHAR	255	NOT NULL	Description of the action	Admin resolved the disruption on CS-1002 via 'Hold at current hub until conditions clear'
4	timestamp	DATETIME	—	DEFAULT current date-time (UTC)	Date and time the action happened	2026-09-07 07:15:49

Relationships

users 1 : N shipments (assigned_operator_id); cargo_types 1 : N shipments; hubs 1 : N shipments; shipments 1 : N disruptions; disruptions 1 : N route_solutions (deleted together

with their disruption); users 1 : N notifications, messages (as sender and as receiver), activity_log entries and resolved disruptions (resolved_by).

3.3 User Interface Design

The user interface is built with server-rendered Jinja2 templates on Bootstrap 5, extended with a custom design system (app/static/css/style.css). Every signed-in page shares one application shell: a collapsible dark sidebar for navigation with live counts, and a top bar with the theme switch, a notification bell and the account menu. The design principles applied to every screen are:

- **Consistent design system** — Inter typeface, an indigo and teal colour palette, soft shadows and rounded cards. Risk levels always use the same colours: green for LOW, amber for AT-RISK, orange for MEDIUM and red for HIGH.
- **Light and dark themes** — switchable from the top bar, defaulting to the operating-system setting; charts and map tiles adapt to the active theme.
- **Responsive layout** — the sidebar collapses to icons on desktop and becomes a slide-in drawer on phones, so no page scrolls sideways even at a 390-pixel phone width.
- **Feedback and motion** — buttons show a spinner while a form is submitting, KPI numbers count up and new alerts slide in as toasts; motion is switched off automatically for users who prefer reduced motion.

Admin Panel Design

The admin panel is the control centre of the system and is available only to administrators. It is organised as tabs — Overview, Users, Shipments, Base Cities (hubs), Cargo Types, Reports, System Config and Activity Log — so that every administrative task is one click away. The screenshots below show the main screens of the admin panel.

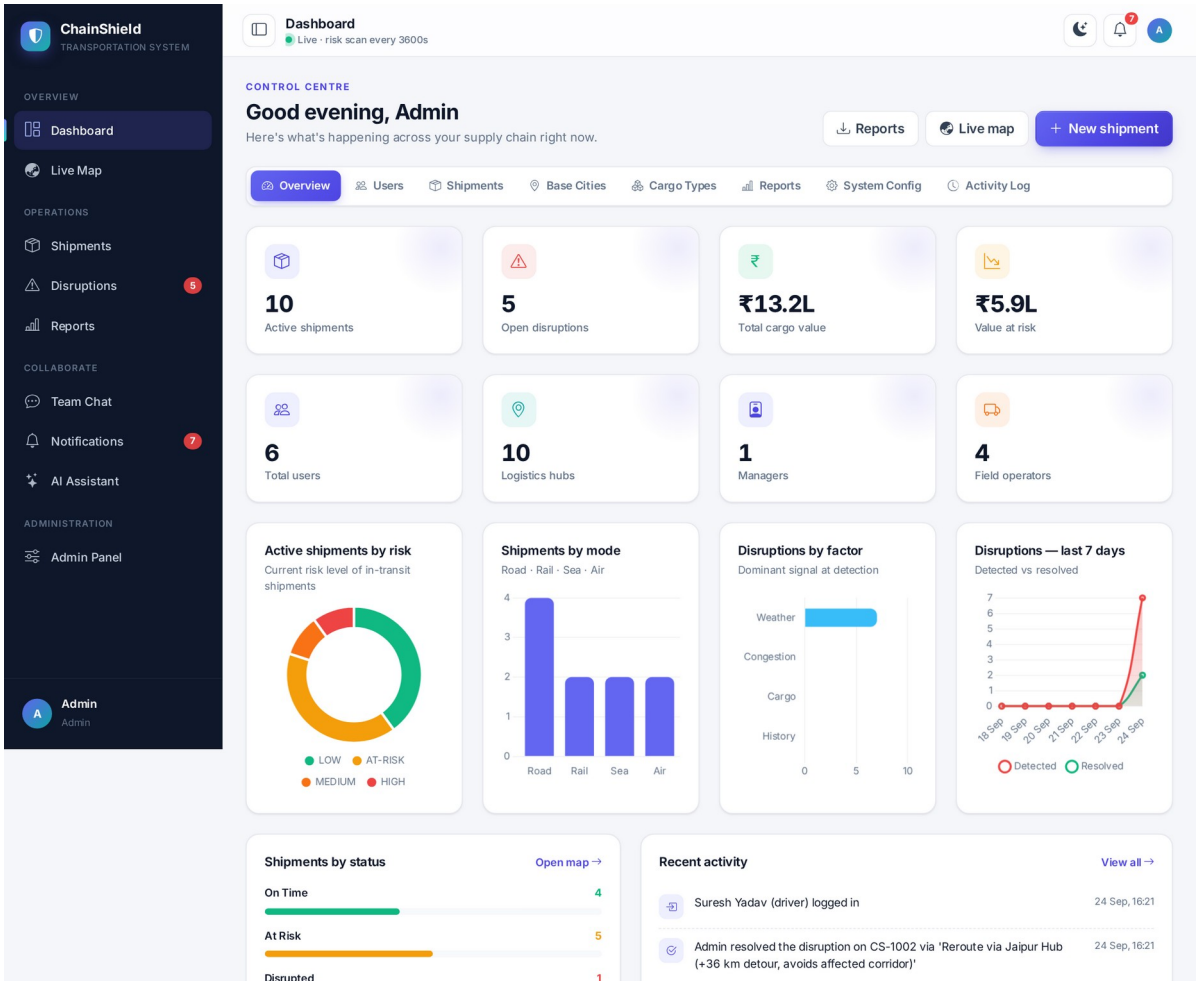


Fig 3.3 — Admin Panel: Overview Dashboard

The overview dashboard shows KPI cards for active shipments, open disruptions, cargo value at risk and total users, four live Chart.js charts (shipments by risk, by transport mode, disruptions by factor and disruptions over the last 7 days) and the recent activity feed. All figures refresh automatically after every risk scan.

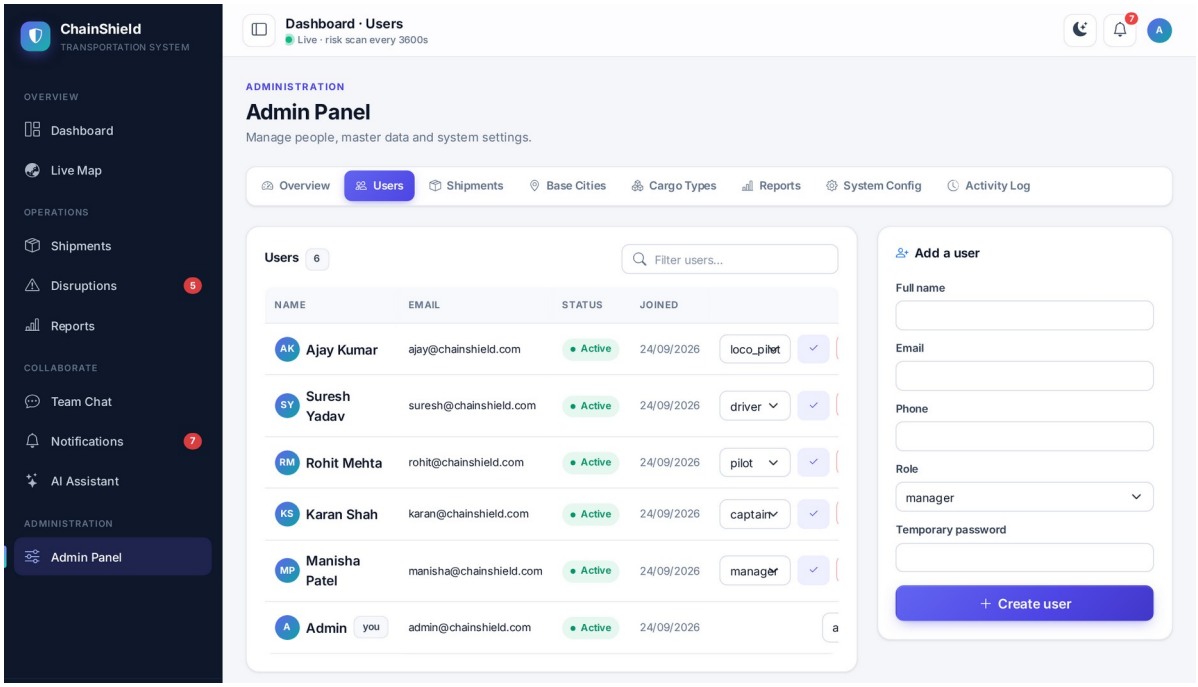


Fig 3.4 — Admin Panel: User Management

The Users tab lists every account with its role and status. The admin can create a new user with a temporary password, change a role, and activate or deactivate an account; a deactivated user is logged out immediately.

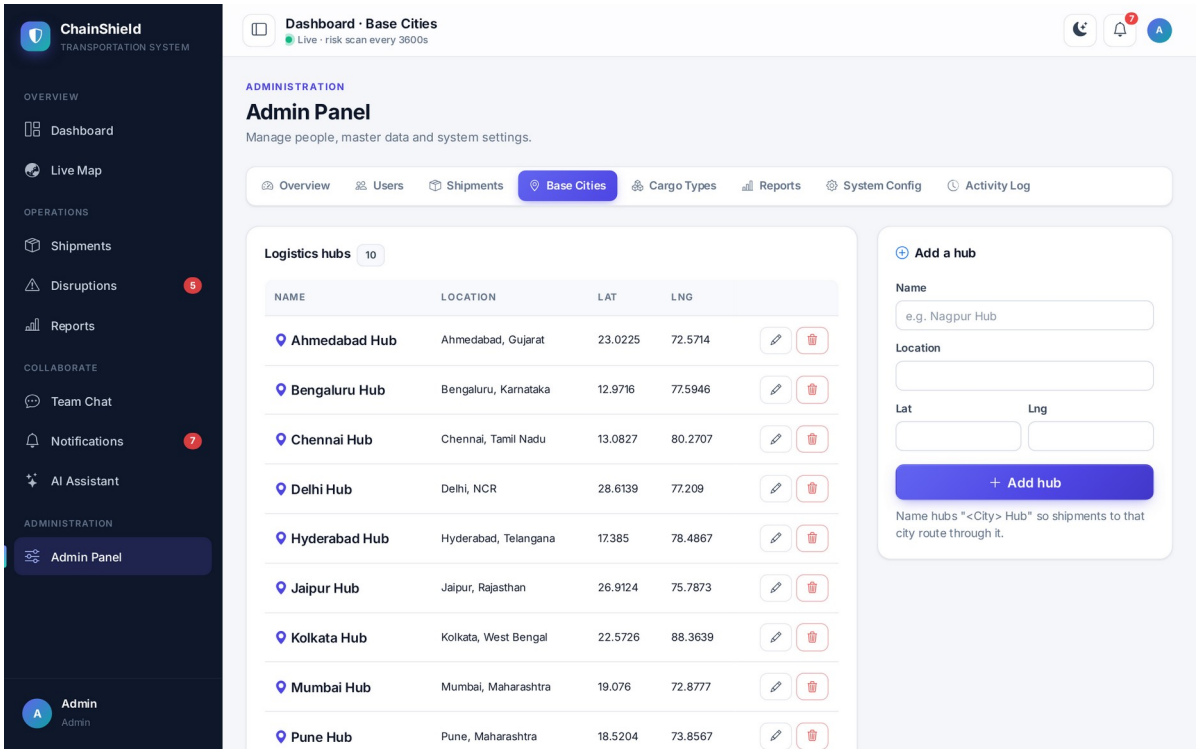


Fig 3.5 — Admin Panel: Logistics Hubs (Base Cities)

The Base Cities tab maintains the hubs used by the route solver. Each hub has a name, a location and latitude/longitude, which are used to plot the hub on the map and to calculate detour distances.

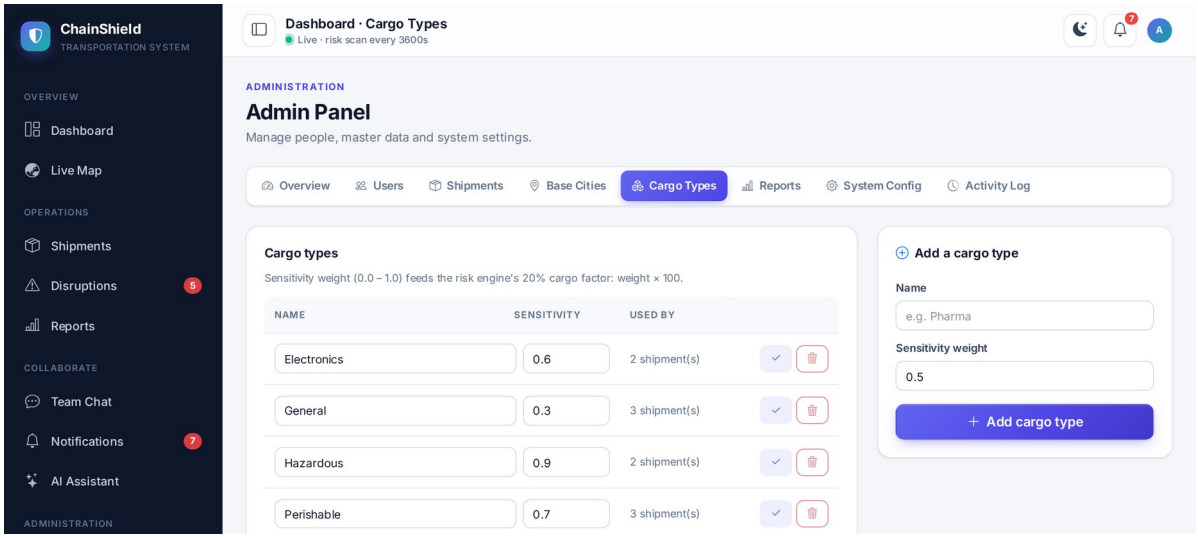


Fig 3.6 — Admin Panel: Cargo Types

The Cargo Types tab maintains the cargo categories and their sensitivity weight between 0.0 and 1.0. A weight outside this range is rejected by server-side validation.

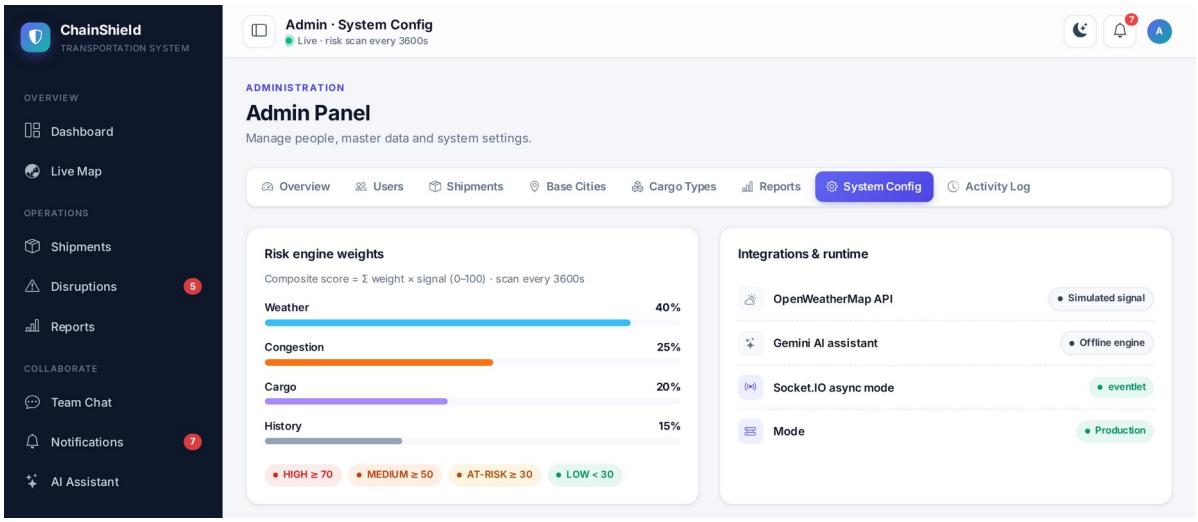


Fig 3.7 — Admin Panel: System Configuration

The System Config tab shows the current risk-engine weights and thresholds, and the status of each integration (OpenWeatherMap, Gemini AI, Socket.IO and demo mode).

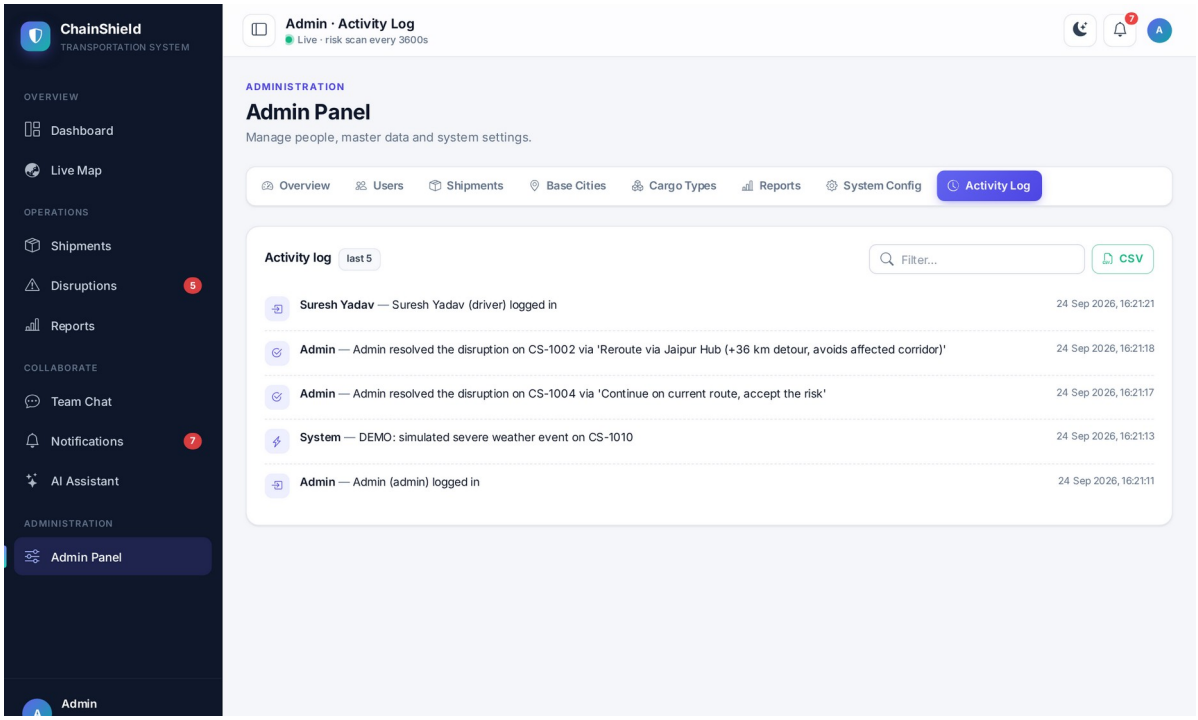


Fig 3.8 — Admin Panel: Activity Audit Log

The Activity Log tab is a searchable, time-ordered record of logins, administrative changes and disruption decisions, which provides accountability for every action taken in the system.

3.4 Report Design

The Reports module is available to administrators and managers. All reports are shown on one screen (Fig 3.9) and are filtered by a From – To date range at the top of the page. Three of the reports can be exported as CSV files that open directly in Microsoft Excel. The design of the reports screen is shown below, followed by the layout of each report.

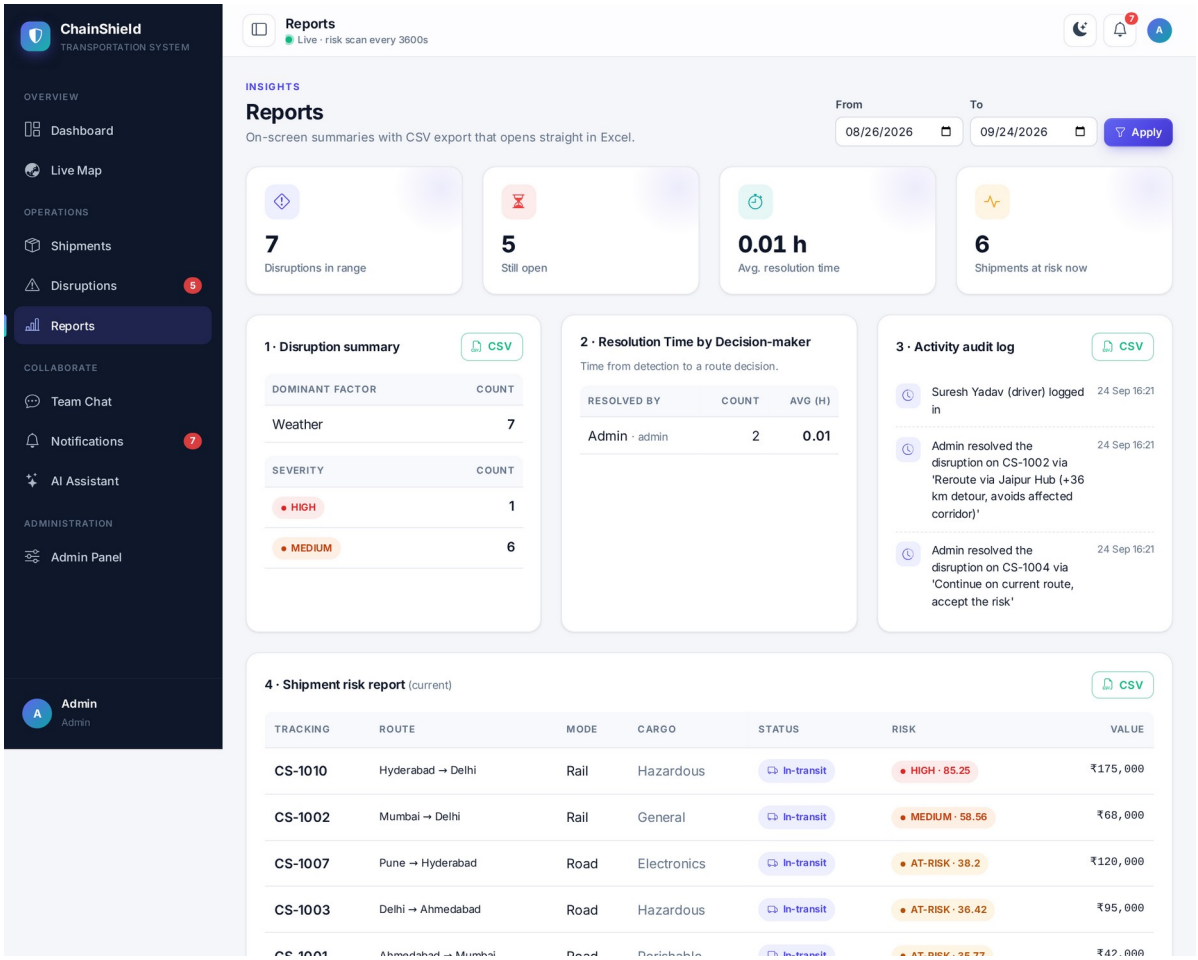


Fig 3.9 — Report Design: Reports Screen with Date Filter and CSV Export

Report 1 — Disruption Summary Report (CSV)

On screen this report shows the number of disruptions by dominant factor and by severity for the selected period. The CSV export lists every disruption with its decision details, so management can see which factors cause most disruptions and how each one was handled. Its layout is:

Table 3.10 — Layout of Disruption Summary Report

Column	Description	Sample Value
ID	Disruption number	1
Shipment	Tracking number	CS-1003
Dominant Factor	Weather / Congestion / Cargo / History	Cargo
Severity	MEDIUM or HIGH	HIGH
Risk Score	Score at detection or last escalation	79.69
Status	Open / Resolved	Resolved
Detected (UTC)	Detection date and time	2026-10-10 20:06
Resolved (UTC)	Decision date and time	2026-10-11 05:45
Resolution Time (h)	Hours from detection to decision	9.63
Resolved By	Manager / admin	Admin
Chosen Option	Route option applied	Reroute via Surat Hub (+370 km detour, avoids affected corridor)

Report 2 — Resolution Time by Decision-maker

This on-screen report shows, for each manager or admin, how many disruptions were resolved in the period and the average time taken from detection to a route decision. It measures how quickly the team responds to risk.

Table 3.11 — Layout of Resolution Time Report (sample rows)

Resolved By	Count	Average Time (hours)
Manisha Patel	4	0.61
Admin	2	1.12

Report 3 — Activity Audit Log (CSV)

A chronological record of logins, administrative actions, shipment changes and disruption resolutions for the selected period, used for accountability and review.

Table 3.12 — Layout of Activity Audit Log Report (sample row)

Timestamp (UTC)	User	Action
2026-09-07 07:15:49	Admin	Admin resolved the disruption on CS-1002 via 'Hold at current hub until conditions clear'

Report 4 — Shipment Risk Report (CSV)

This report lists the current risk of every shipment, so a manager can see at a glance which shipments, and how much cargo value, are at risk right now. Its layout is:

Table 3.13 — Layout of Shipment Risk Report

Column	Description	Sample Value
Tracking No	Shipment tracking number	CS-1004
Origin / Destination	Route of the shipment	Bengaluru / Chennai
Mode	Road / Rail / Sea / Air	Air
Cargo Type	Cargo category	Electronics
Status	Current shipment status	In-transit
Risk Level / Score	Result of the last scan	MEDIUM / 53.20
Cargo Value (INR)	Declared value	150000
Operator	Assigned field operator	Rohit Mehta
Latitude / Longitude	Current position	13.08 / 80.27

4. DEVELOPMENT

4.1 Coding Standards

The following coding standards were followed throughout the development of ChainShield. For every standard, a short explanation, an example, and a screenshot of the actual source code where the standard is applied are given.

1. PEP 8 Naming, Type Hints and Docstrings

All Python code follows PEP 8: 4-space indentation, snake_case for functions and variables, PascalCase for classes and UPPER_CASE for constants. Functions carry type hints and a docstring, so every member of the group can read and review another member's code without explanation.

Example: compute_risk_score() and classify_risk() in app/services/risk_engine.py; the constant ALERT_LEVELS; the class RouteSolution in app/models.py.

chainshield-mca/app/services/risk_engine.py — lines 32-52

```
32 def compute_risk_score(weather: float, congestion: float, cargo: float, history: float) -> float:
33     """Return a composite 0-100 risk score from the four weighted factors."""
34     cfg = current_app.config
35     score = (
36         weather * cfg["RISK_WEIGHT_WEATHER"]
37         + congestion * cfg["RISK_WEIGHT_CONGESTION"]
38         + cargo * cfg["RISK_WEIGHT_CARGO"]
39         + history * cfg["RISK_WEIGHT_HISTORY"]
40     )
41     return round(score, 2)
42
43
44 def classify_risk(score: float) -> str:
45     if score >= 70:
46         return "HIGH"
47     if score >= 50:
48         return "MEDIUM"
49     if score >= 30:
50         return "AT-RISK"
51     return "LOW"
```

Fig 4.1 — Naming, type hints and docstring in the risk engine

2. Application Factory Pattern

The Flask application is never created at import time; create_app() builds it from a named configuration (development, testing or production). This lets the same code run on a laptop, inside the automated tests and inside Docker, each with its own settings.

Example: create_app("testing") gives the test suite a fresh in-memory database, while Docker calls create_app("production").

chainshield-mca/app/__init__.py — lines 15-26

```python
def create_app(config_name=None):
    config_name = config_name or os.environ.get("FLASK_CONFIG", "development")

    app = Flask(__name__, instance_relative_config=True)
    app.config.from_object(config_by_name[config_name])

    os.makedirs(app.instance_path, exist_ok=True)

    register_extensions(app)
    register_blueprints(app)
    register_cli(app)
```

Fig 4.2 — create_app() in app/__init__.py

3. Modular Blueprint Structure

Each feature lives in its own Flask blueprint with its own URL prefix, routes and templates. A feature can therefore be changed, tested or disabled without touching the others.

Example: shipments_bp serves everything under /shipments, disruptions_bp under /disruptions and chat_bp under /chat.

chainshield-mca/app/__init__.py — lines 129-144

```python
def register_blueprints(app):
    from app.blueprints.main.routes import main_bp
    from app.blueprints.auth.routes import auth_bp
    from app.blueprints.shipments.routes import shipments_bp
    from app.blueprints.disruptions.routes import disruptions_bp
    from app.blueprints.notifications.routes import notifications_bp
    from app.blueprints.chat.routes import chat_bp
    from app.blueprints.assistant.routes import assistant_bp

    app.register_blueprint(main_bp)
    app.register_blueprint(auth_bp, url_prefix="/auth")
    app.register_blueprint(shipments_bp, url_prefix="/shipments")
    app.register_blueprint(disruptions_bp, url_prefix="/disruptions")
    app.register_blueprint(notifications_bp, url_prefix="/notifications")
    app.register_blueprint(chat_bp, url_prefix="/chat")
    app.register_blueprint(assistant_bp, url_prefix="/assistant")
```

Fig 4.3 — Blueprint registration in app/__init__.py

4. Single Source of Configuration — No Hard-coded Secrets

Risk weights, the scan interval and model names are defined once in app/config.py; API keys and the secret key are read from a .env file. The .env file is excluded from Git and Docker, so no password or key is ever committed to the repository.

Example: RISK_WEIGHT_WEATHER = 0.40 is read by the risk engine; GEMINI_API_KEY = os.environ.get("GEMINI_API_KEY", "").

chainshield-mca/app/config.py — lines 7-21

```python
class BaseConfig:
    SECRET_KEY = os.environ.get("SECRET_KEY", "dev-secret-key-change-me")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    PERMANENT_SESSION_LIFETIME = timedelta(hours=8)

    # Risk engine
    RISK_SCAN_INTERVAL_SECONDS = int(os.environ.get("RISK_SCAN_INTERVAL_SECONDS", 60))
    RISK_WEIGHT_WEATHER = 0.40
    RISK_WEIGHT_CONGESTION = 0.25
    RISK_WEIGHT_CARGO = 0.20
    RISK_WEIGHT_HISTORY = 0.15

    # External APIs (optional — services fall back to simulated data if unset)
    OPENWEATHER_API_KEY = os.environ.get("OPENWEATHER_API_KEY", "")
    GEMINI_API_KEY = os.environ.get("GEMINI_API_KEY", "")
```

Fig 4.4 — Centralised settings in app/config.py

5. ORM Models Instead of Raw SQL

All nine tables are Python classes in app/models.py, and every query goes through Flask-SQLAlchemy. Queries are parameterised automatically, which prevents SQL injection, and the database can be moved to PostgreSQL by changing only DATABASE_URL.

Example: User.query.filter_by(email=form.email.data).first() instead of a hand-written SELECT statement.

chainshield-mca/app/models.py — lines 15-34

```python
class User(db.Model, UserMixin):
    __tablename__ = "users"

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    email = db.Column(db.String(100), unique=True, nullable=False, index=True)
    password_hash = db.Column(db.String(255), nullable=False)
    role = db.Column(db.String(20), nullable=False, default="manager")
    # roles: admin | manager | captain | pilot | driver | loco_pilot
    phone = db.Column(db.String(20))
    # Overrides UserMixin.is_active: deactivated users can't log in (and are
    # logged out on their next request — see load_user in app/__init__.py).
    is_active = db.Column(db.Boolean, nullable=False, default=True, server_default="1")
    created_at = db.Column(db.DateTime, default=datetime.utcnow)

    def set_password(self, raw_password):
        self.password_hash = generate_password_hash(raw_password)

    def check_password(self, raw_password):
        return check_password_hash(self.password_hash, raw_password)
```

Fig 4.5 — The User model with password hashing helpers

6. Server-side Form Validation

Every form is validated on the server with Flask-WTF before anything is saved, because browser-side checks can be bypassed. Business rules that span two fields are written in a validate() method of the form class.

Example: ShipmentForm rejects a shipment whose destination equals its origin and any latitude outside −90 to 90.

chainshield-mca/app/forms.py — lines 10-30

```python
class ShipmentForm(FlaskForm):
    tracking_no = StringField("Tracking No.", validators=[Optional(), Length(max=30)],
                              description="Leave blank to auto-generate (CS-xxxx).")
    origin = SelectField("Origin city", validators=[DataRequired()])
    destination = SelectField("Destination city", validators=[DataRequired()])
    transport_mode = SelectField("Transport mode", choices=TRANSPORT_MODES, validators=[DataRequired()])
    cargo_type_id = SelectField("Cargo type", coerce=int, validators=[DataRequired()])
    cargo_value = FloatField("Cargo value (₹)", validators=[Optional(), NumberRange(min=0)], default=0)
    status = SelectField("Status", choices=SHIPMENT_STATUSES, default="In-transit")
    assigned_operator_id = SelectField("Assigned operator", coerce=int)
    current_lat = FloatField("Current latitude", validators=[Optional(), NumberRange(min=-90, max=90)],
                             description="Leave blank to start at the origin hub.")
    current_lng = FloatField("Current longitude", validators=[Optional(), NumberRange(min=-180, max=180)])
    submit = SubmitField("Save shipment")

    def validate(self, extra_validators=None):
        ok = super().validate(extra_validators)
        if ok and self.origin.data == self.destination.data:
            self.destination.errors.append("Destination must differ from origin.")
            return False
        return ok
```

Fig 4.6 — ShipmentForm in app/forms.py

7. Security by Default — Role Decorators

Access rules are written once as decorators and placed on top of each route, so no page can forget its permission check. Together with CSRF tokens on every form, Werkzeug password hashing and Jinja2 auto-escaping, this protects against the common web attacks.

Example: @manager_required on /shipments/new returns 403 Forbidden to drivers, pilots, captains and loco pilots.

chainshield-mca/app/blueprints/shipments/routes.py — lines 18-30

```python
def manager_required(view):
    """Admins and managers can create, edit and delete shipments."""

    @wraps(view)
    @login_required
    def wrapped(*args, **kwargs):
        if current_user.role not in ("admin", "manager"):
            abort(403)
        return view(*args, **kwargs)

    return wrapped
```

Fig 4.7 — manager_required decorator in the shipments blueprint

8. Template Inheritance and Reusable Macros

base.html defines the sidebar, top bar and scripts once, and every page only fills its content block. Repeated interface pieces are Jinja2 macros, so a design change is made in exactly one place.

Example: {{ risk_badge(s.risk_level, s.risk_score) }} draws the same coloured risk pill on the dashboard, shipment list and map pop-ups.

chainshield-mca/app/templates/_macros.html — lines 1-7

```
{% macro risk_badge(level, score=None) -%}
  {% if level %}
  <span class="pill risk-{{ level }}">{{ level }}{% if score is not none %} · {{ score }}{% endif %}</span>
  {%- else -%}
  <span class="pill status-pill">Not scanned</span>
  {%- endif %}
{%- endmacro %}
```

Fig 4.8 — risk_badge macro in app/templates/_macros.html

9. Version Control and Pinned Dependencies

Git records every feature with a descriptive commit message, and every library version is pinned in requirements.txt. The Dockerfile installs exactly these versions on Python 3.10, so the application behaves identically on every machine.

Example: Commit “Add admin CRUD, reports with CSV export, demo mode and map zones”; Flask==3.0.3 in requirements.txt.

chainshield-mca/requirements.txt — lines 1-13

```
Flask==3.0.3
Flask-SQLAlchemy==3.1.1
Flask-Login==0.6.3
Flask-SocketIO==5.3.6
Flask-WTF==1.2.1
APScheduler==3.10.4
python-dotenv==1.0.1
Werkzeug==3.0.3
requests==2.32.3
google-generativeai==0.7.2
gunicorn==22.0.0
eventlet==0.36.1
simple-websocket==1.0.0
```

Fig 4.9 — Pinned dependencies in requirements.txt

4.2 Implementation Details

Risk Engine (app/services/risk_engine.py)

APScheduler runs a scan every 60 seconds (configurable). Each in-transit shipment is scored with the formula: score = 0.40 × weather + 0.25 × congestion + 0.20 × cargo sensitivity + 0.15 × lane history, where each factor is on a scale of 0 – 100. A score of 70 or more is HIGH, 50 or more is MEDIUM, 30 or more is AT-RISK and anything lower is LOW. AT-RISK shipments are only watched; a disruption is opened at MEDIUM and escalated, with re-planned options, if the shipment reaches HIGH. Weather comes from OpenWeatherMap when an API key is configured; otherwise a simulated signal is used. Lane history grows with disruptions on the same origin–destination lane in the last 90 days. In demo mode each scan also moves shipments towards their destination hub, marking them Delivered on arrival.

Route Solver (app/services/route_solver.py)

For each disruption the solver builds these candidates: hold at the nearest hub, continue on the current route (baseline), reroute through the two hubs with the smallest detour (using the haversine great-circle distance), and a mode switch for the remaining leg (for example Road → Rail). Each option receives an extra time, an extra cost (₹, from per-kilometre mode rates) and a residual risk, and is ranked by 0.60 × risk + 0.25 × normalised time + 0.15 × normalised cost. Option 1 is the recommendation.

Real-time Layer (Flask-SocketIO)

Every logged-in browser joins a personal room (user_<id>). Notifications and chat messages are saved in the database and pushed to the right room; broadcast events such as shipments_updated and disruption_new make the live map, dashboards and charts refresh themselves without polling.

AI Assistant (app/services/assistant.py)

The user's question and a snapshot of the live data that the user is allowed to see (shipments, risk, open disruptions and their ranked options) are sent to Google Gemini. Without a key or an internet connection, a rule-based offline engine answers from the same data, so the feature always works during a demonstration.

Security and Deployment

Passwords are hashed with Werkzeug, sessions are handled by Flask-Login, every form and AJAX call carries a CSRF token, every route checks the user's role, accounts can be deactivated, and nobody can register as admin. The application is packaged as a Docker image on Python 3.10 served by Gunicorn with an eventlet worker (for real WebSockets); the SQLite file is persisted through a Docker volume and secrets are supplied through a .env file that is never committed.

4.3 Output Screens

The screenshots below were captured from the running application with demo data. The admin panel screens are shown in Section 3.3 and the reports screen in Section 3.4.

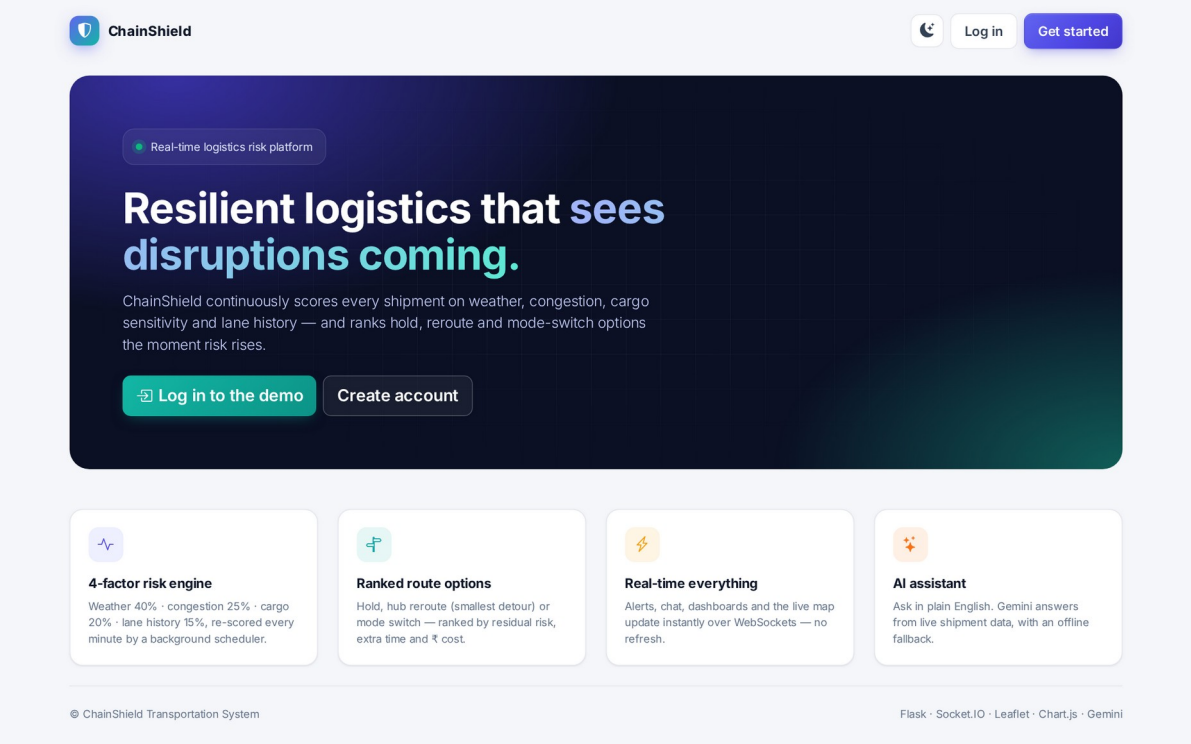


Fig 4.10 — Landing Page

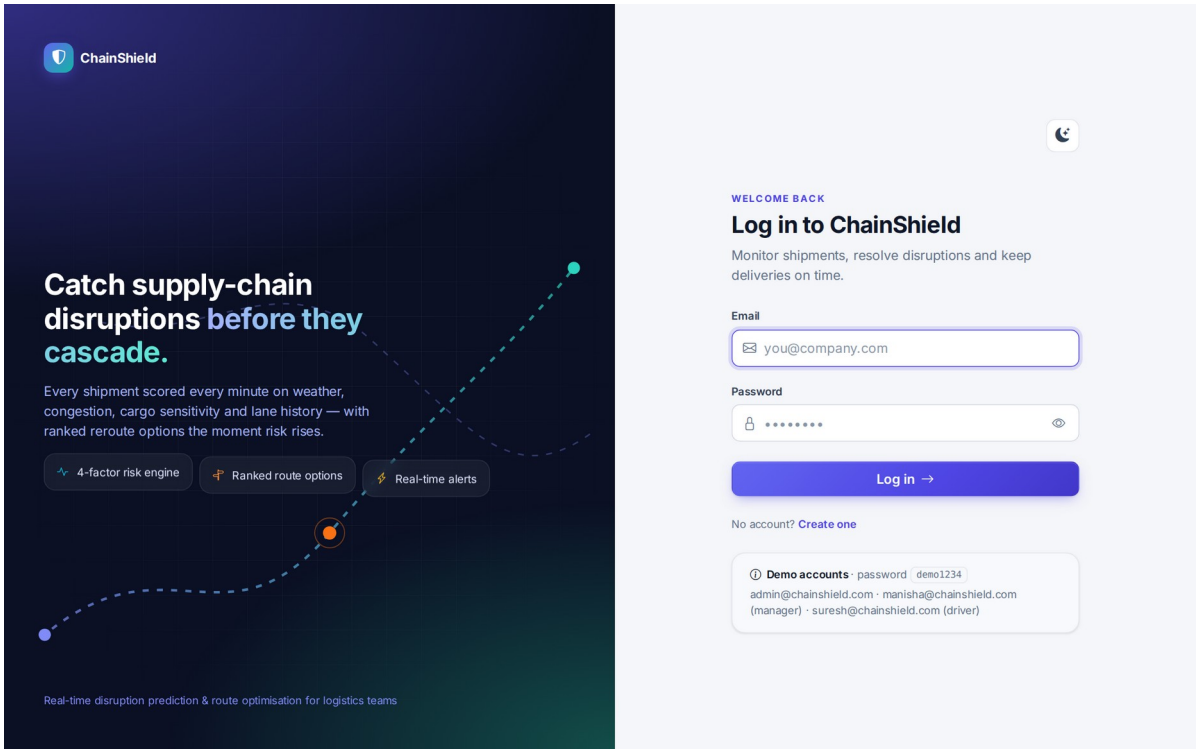


Fig 4.11 — Login Screen

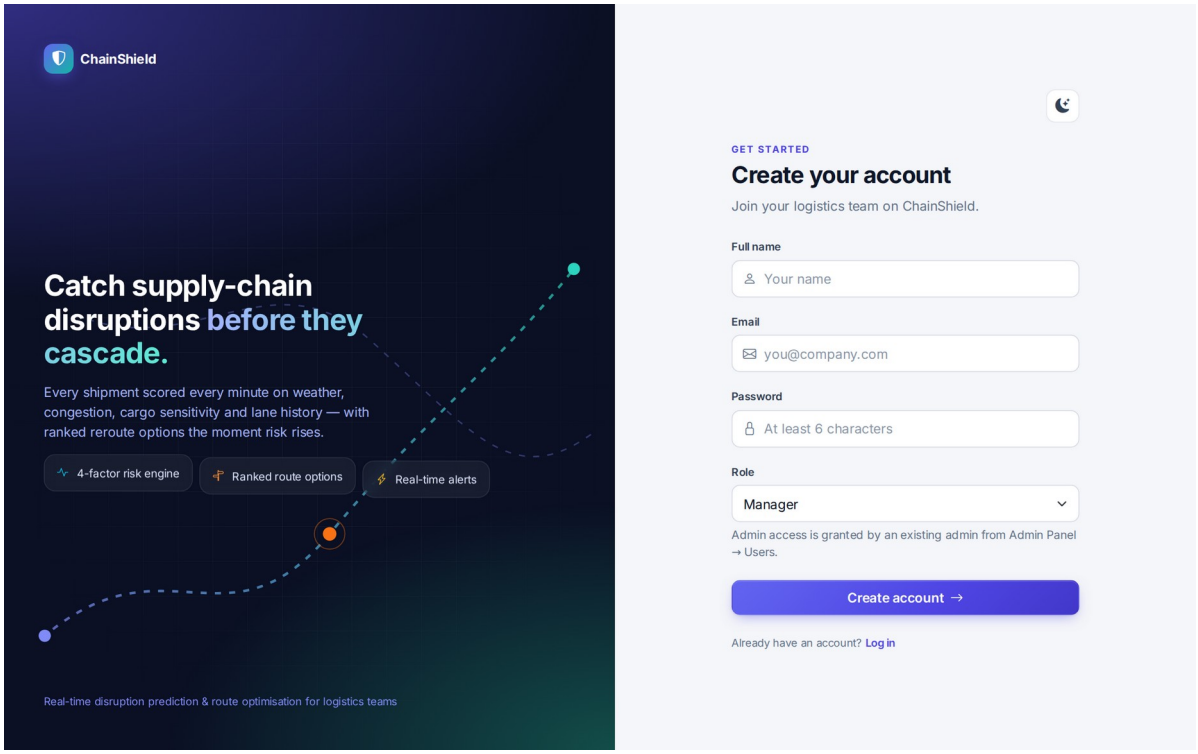


Fig 4.12 — Registration Screen

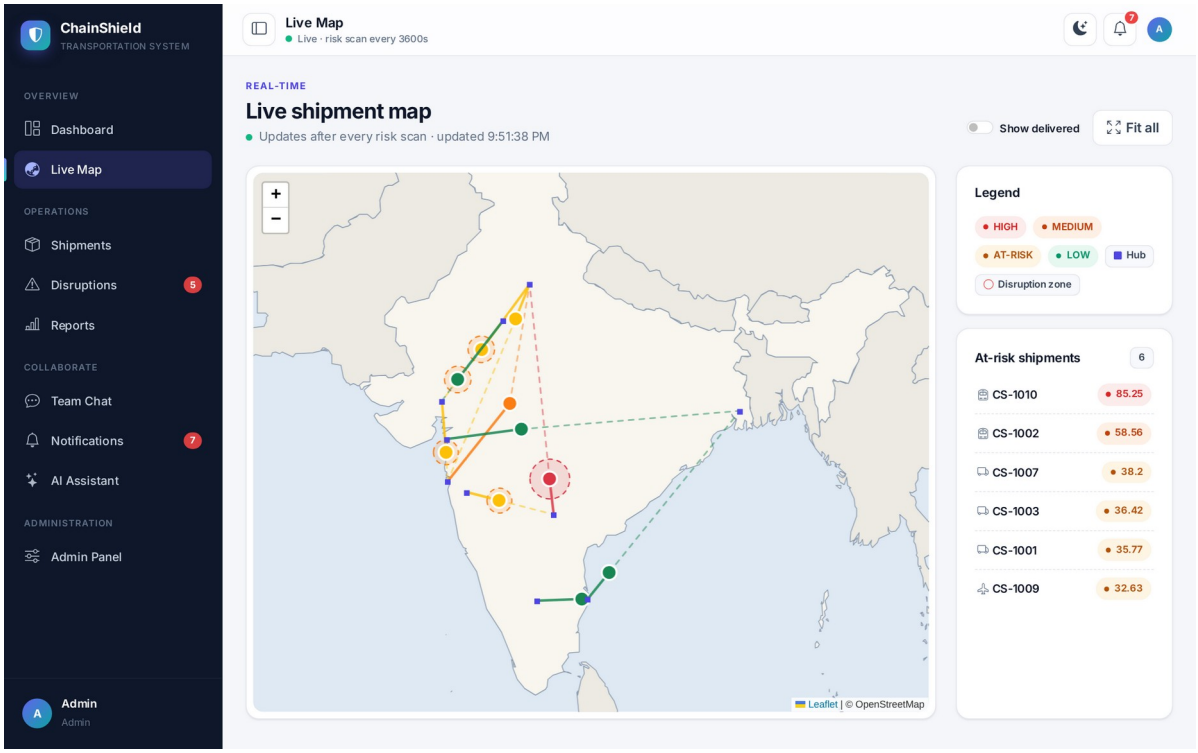


Fig 4.13 — Live Shipment Map with Disruption Zones

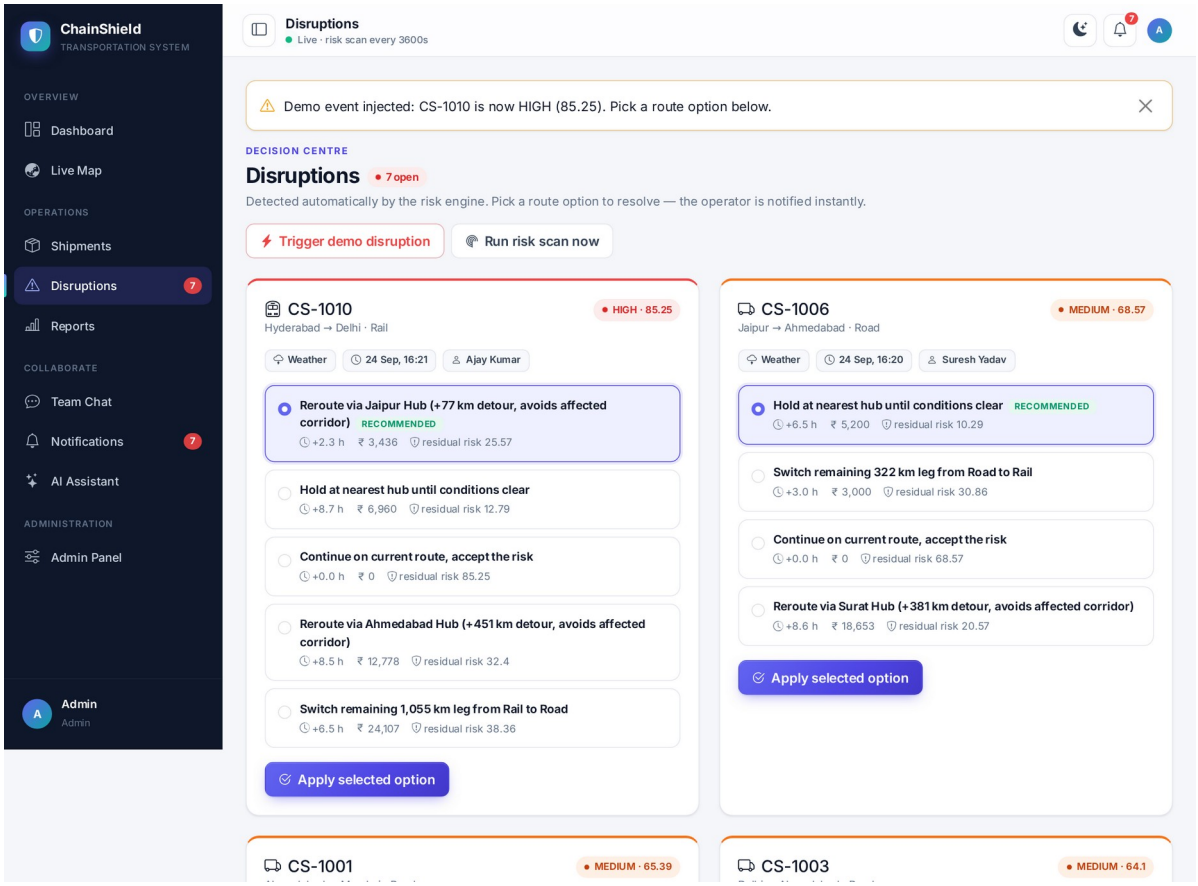


Fig 4.14 — Disruptions with Ranked Route Options

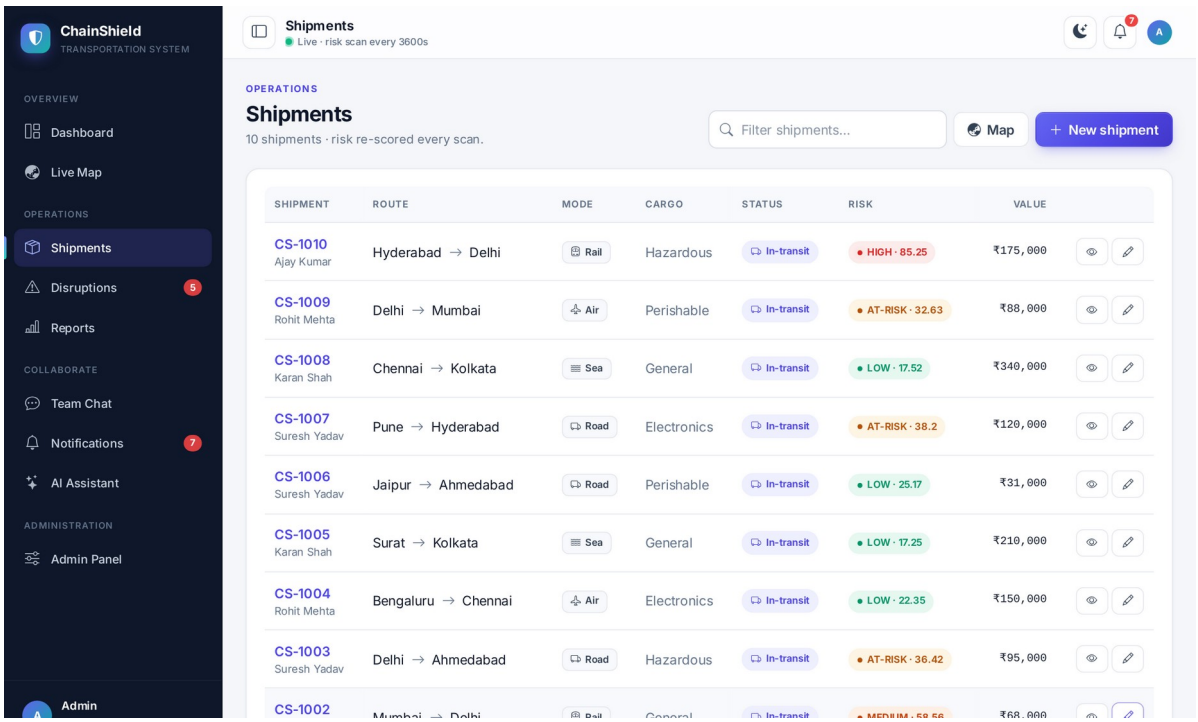


Fig 4.15 — Shipment List

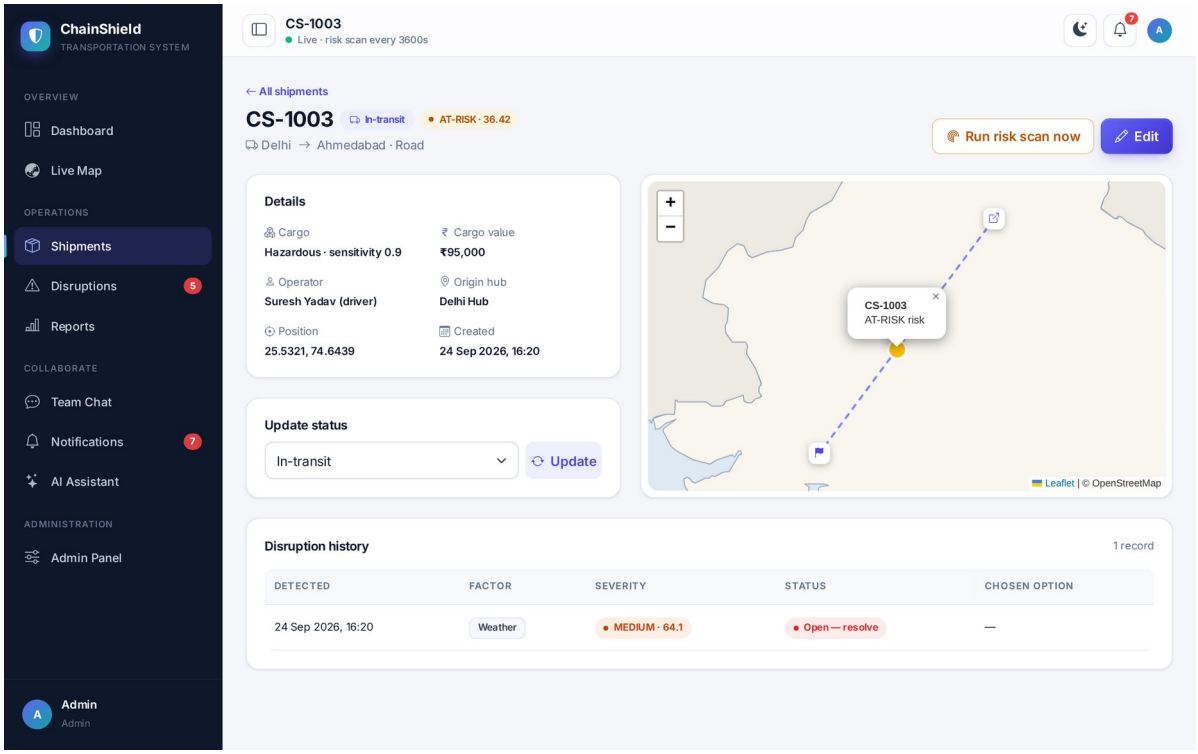


Fig 4.16 — Shipment Detail with Mini-map and History

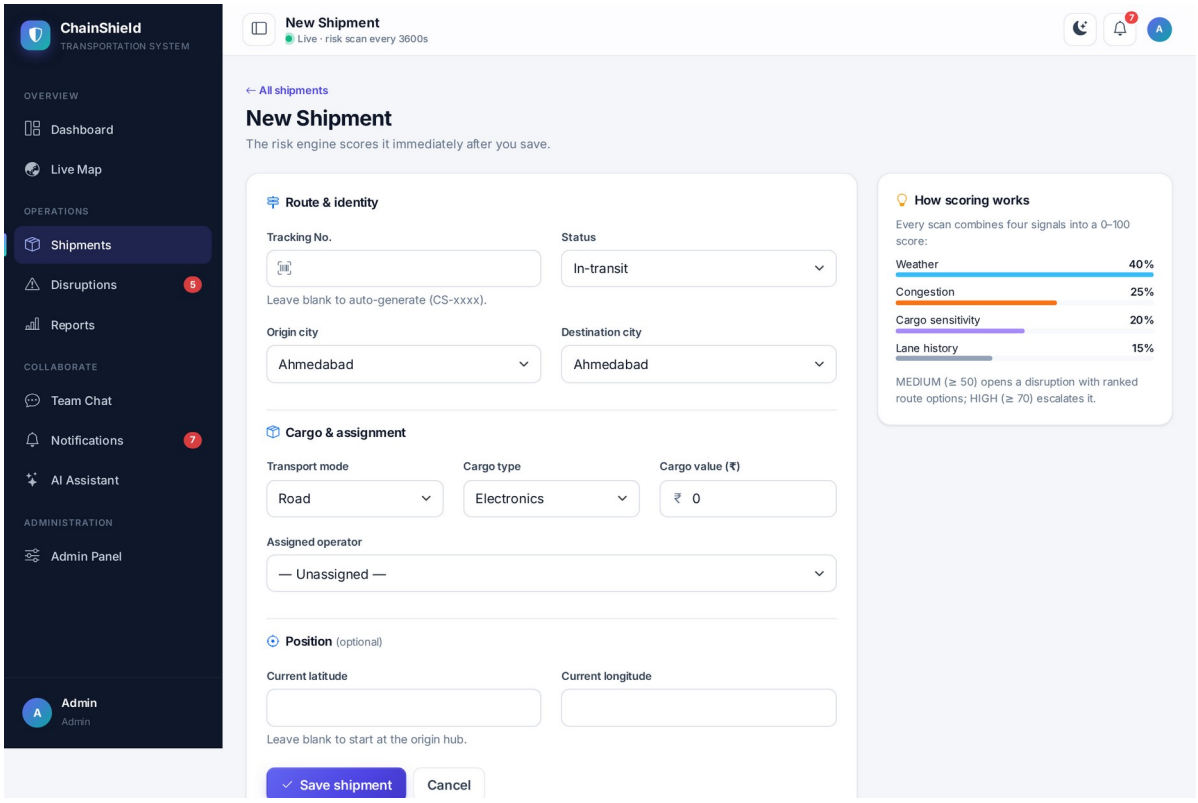


Fig 4.17 — Create Shipment Form

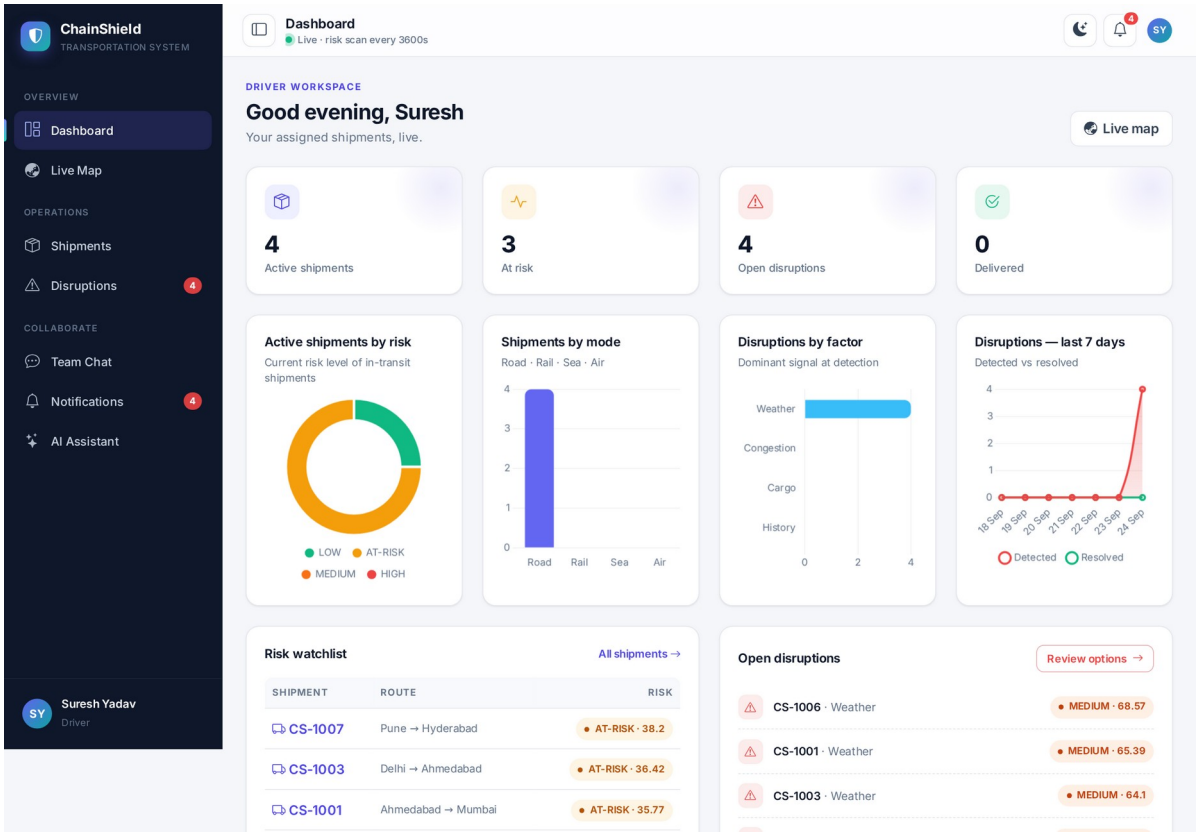


Fig 4.18 — Field Operator (Driver) Dashboard

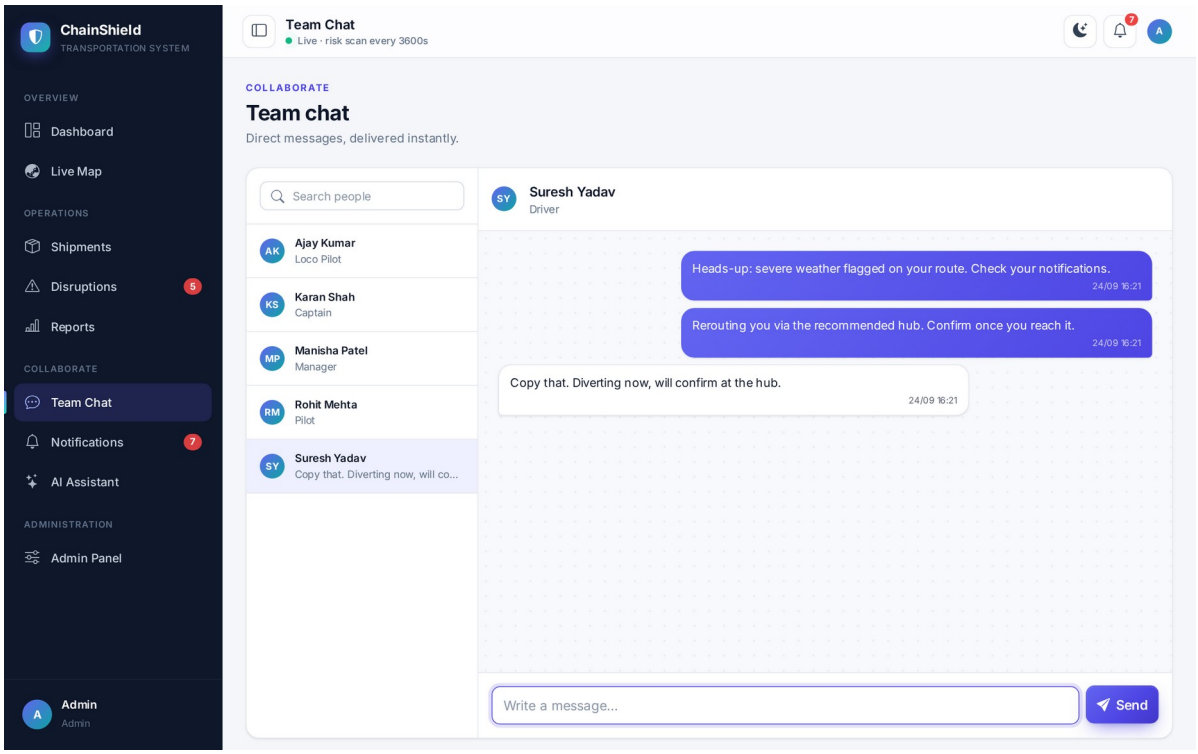


Fig 4.19 — Real-time Team Chat

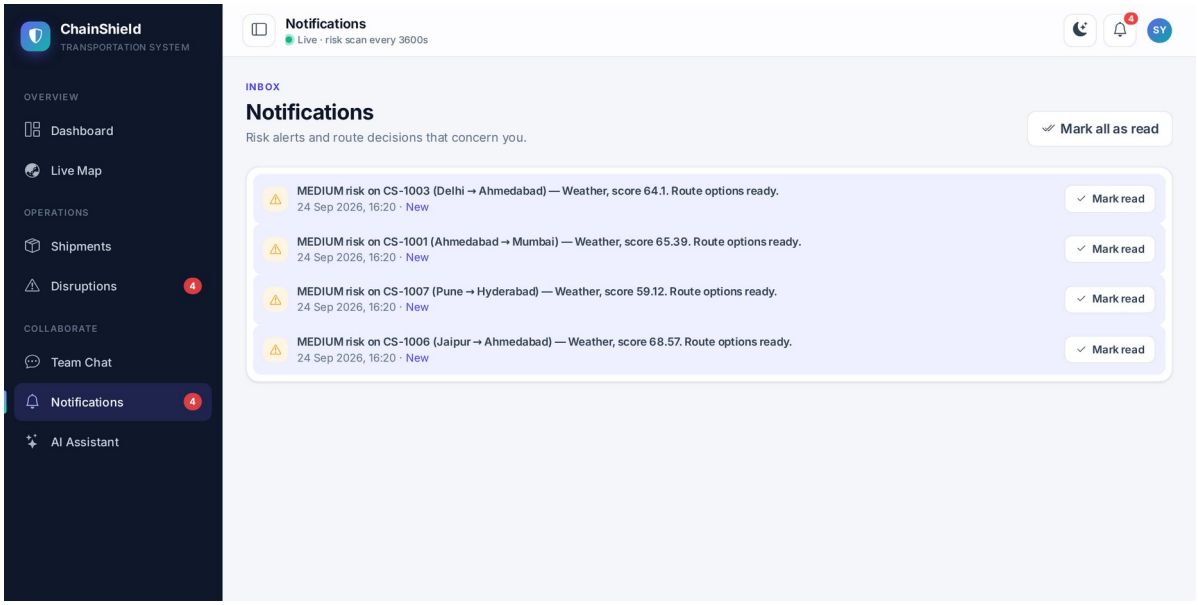


Fig 4.20 — Notification Feed

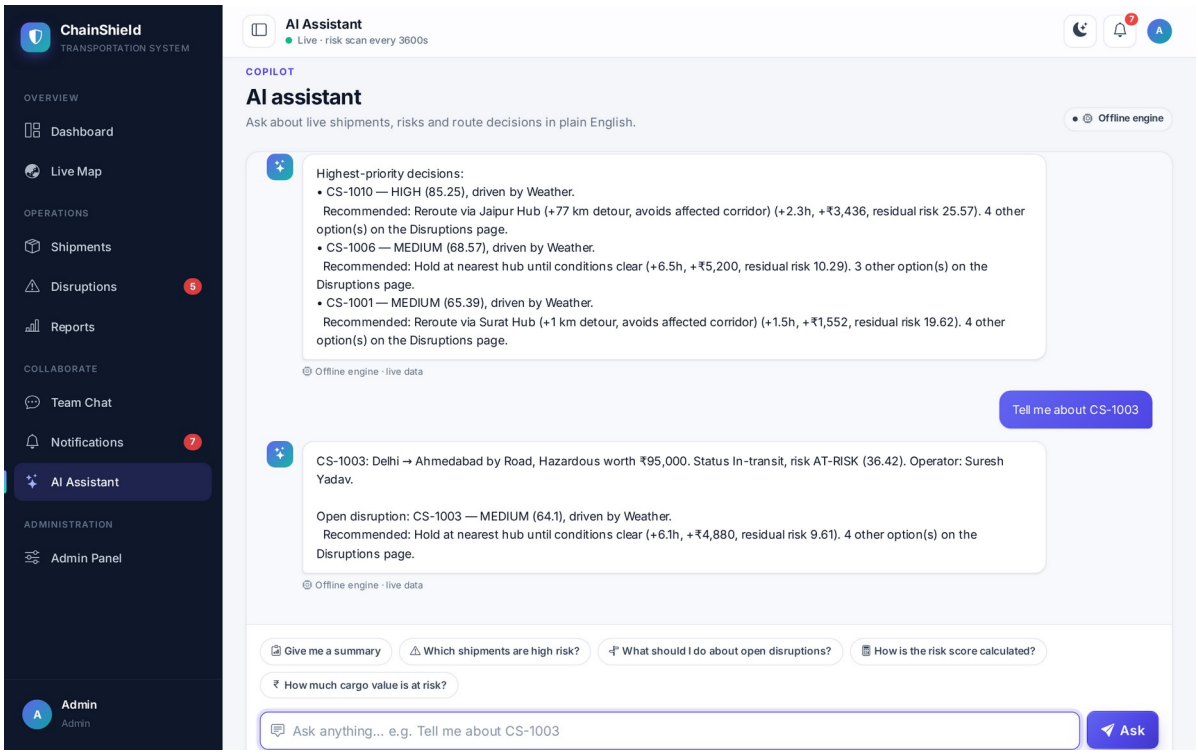


Fig 4.21 — AI Assistant with Suggested Questions

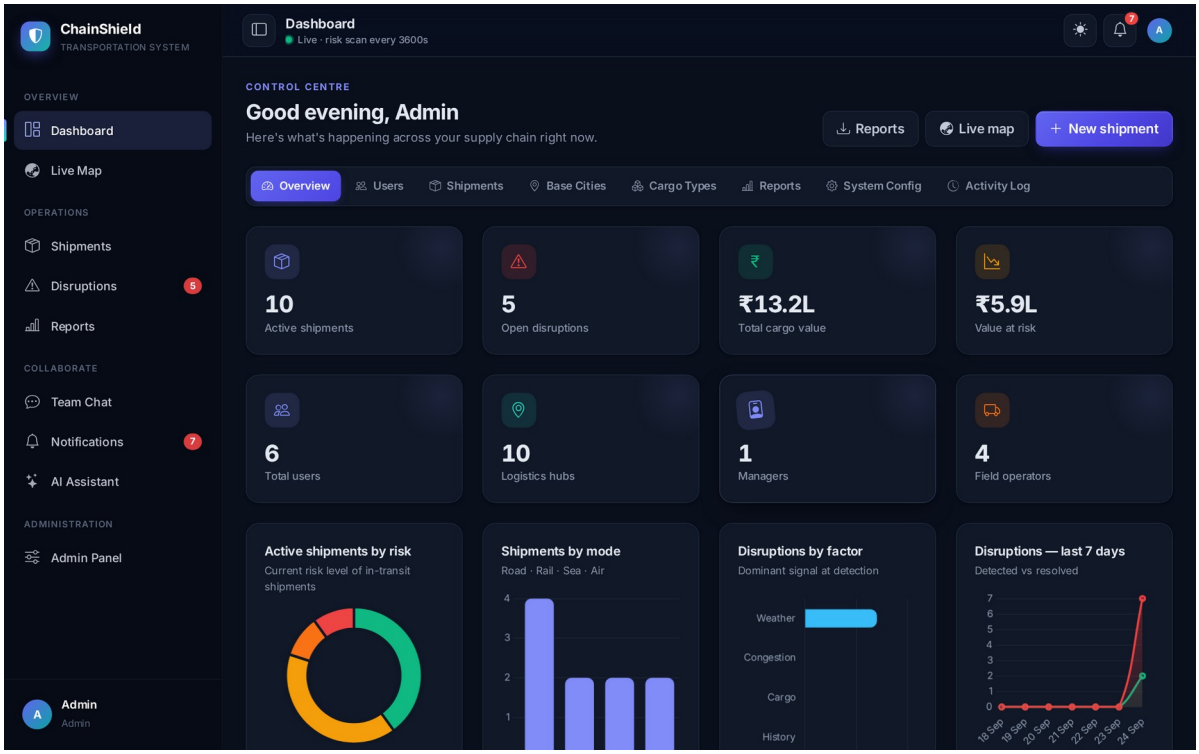


Fig 4.22 — Dark Theme — Dashboard

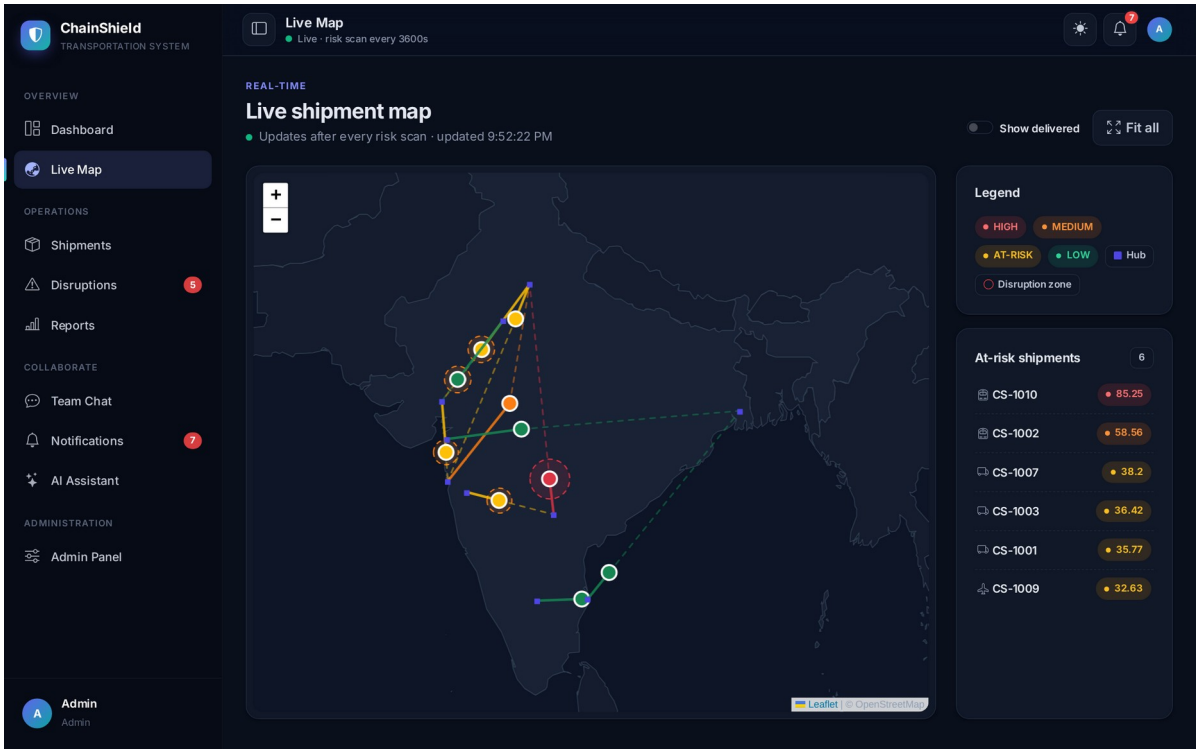


Fig 4.23 — Dark Theme — Live Map

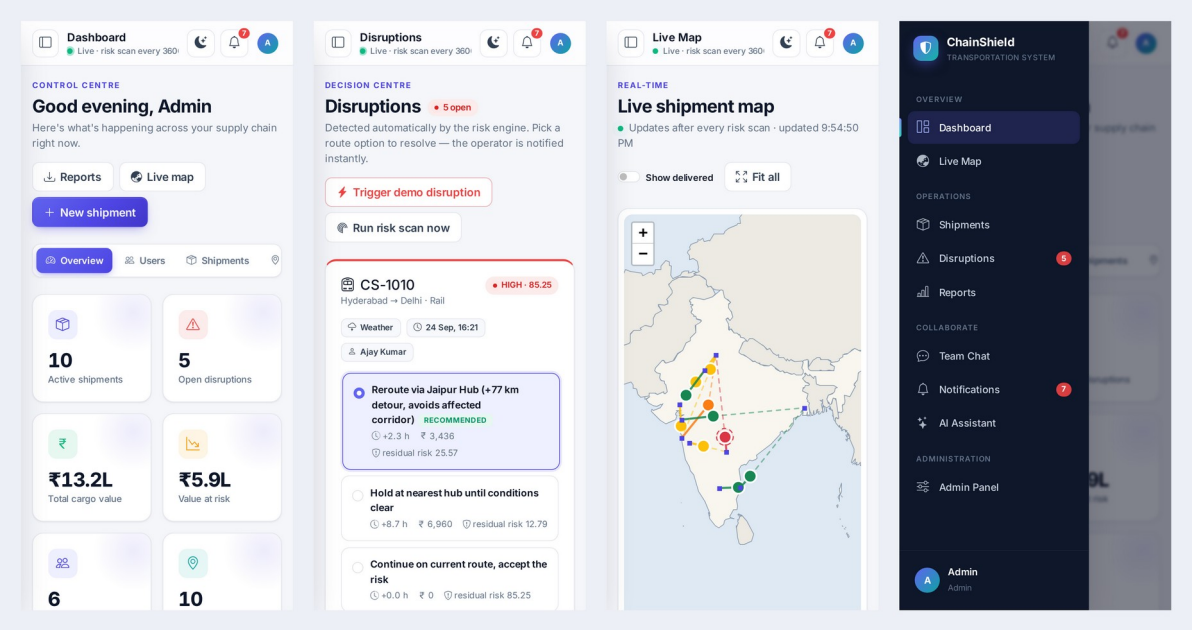


Fig 4.24 — Mobile Views — Dashboard, Disruptions, Live Map and Navigation Drawer

5. TESTING

5.1 Test Cases

Testing combined an automated test script (101 checks run through Flask's test client and the Socket.IO test client, covering every route, role restriction, the risk and route-solver logic, real-time events, chat, the assistant, reports and admin CRUD), manual testing of every screen in the browser, and a Docker deployment test. The representative test cases are described below.

Test Case 1 — User Login

Field	Details
Test ID	TC001
Description	Verify that a registered user can log in with correct credentials.
Pre-conditions	User account exists in the database.
Test Data	email = manisha@chainshield.com, password = demo1234
Expected Result	User is authenticated and redirected to the role-specific dashboard.
Actual Result	User is authenticated and redirected to the role-specific dashboard.
Status	Pass

Test Case 2 — Risk Score Calculation

Field	Details
Test ID	TC002
Description	Verify the risk engine computes the composite score from the 4 factors.
Pre-conditions	Weights configured as 0.40 / 0.25 / 0.20 / 0.15.
Test Data	Weather = 80, Congestion = 60, Cargo = 50, History = 40
Expected Result	Score = 0.40×80 + 0.25×60 + 0.20×50 + 0.15×40 = 63 (MEDIUM)
Actual Result	Score = 0.40×80 + 0.25×60 + 0.20×50 + 0.15×40 = 63 (MEDIUM)
Status	Pass

Test Case 3 — Automatic Disruption & Escalation

Field	Details
Test ID	TC003
Description	Verify a disruption is auto-created at MEDIUM and escalated at HIGH.
Pre-conditions	Scheduler running; in-transit shipment with no open disruption.
Test Data	Scan 1: score 55 (MEDIUM); Scan 2: score 74 (HIGH)
Expected Result	One disruption opened at MEDIUM with ≥ 3 ranked solutions, then escalated to HIGH with re-planned options.
Actual Result	One disruption opened at MEDIUM with ≥ 3 ranked solutions, then escalated to HIGH with re-planned options.
Status	Pass

Test Case 4 — Route Solution & Notification

Field	Details
Test ID	TC004
Description	Verify resolving a disruption notifies the assigned field operator.
Pre-conditions	Open disruption with ranked solutions; manager logged in.
Test Data	Manager selects Option 1 “Reroute via Jaipur Hub”
Expected Result	Disruption becomes Resolved; operator receives a live notification; action logged.
Actual Result	Disruption becomes Resolved; operator receives a live notification; action logged.
Status	Pass

Test Case 5 — Admin CRUD (Cargo Type)

Field	Details
Test ID	TC005
Description	Verify admin can add, edit and delete a cargo type.
Pre-conditions	Admin logged in.
Test Data	name = Pharma, weight 0.9 → 0.85; weight 1.5 (invalid)
Expected Result	Cargo type saved, edited and deleted; weight outside 0.0–1.0 rejected.
Actual Result	Cargo type saved, edited and deleted; weight outside 0.0–1.0 rejected.
Status	Pass

Test Case 6 — Docker Container Startup

Field	Details
Test ID	TC006
Description	Verify the application starts inside Docker.
Pre-conditions	Docker Desktop on Windows 11.
Test Data	docker compose up --build
Expected Result	Image builds; Gunicorn + eventlet starts on port 5000; /health returns ok.
Actual Result	Image builds; Gunicorn + eventlet starts on port 5000; /health returns ok.
Status	Pass

Test Case 7 — Role-Based Access Control

Field	Details
Test ID	TC007
Description	Verify field operators cannot reach admin/manager pages.
Pre-conditions	Driver account with 3 assigned shipments.
Test Data	GET /dashboard/users, /shipments/new, /reports
Expected Result	403 Forbidden for admin/manager pages; 404 for other operators' shipments.
Actual Result	403 Forbidden for admin/manager pages; 404 for other operators' shipments.
Status	Pass

Test Case 8 — Real-time Chat Delivery

Field	Details
Test ID	TC008
Description	Verify a chat message is stored and delivered live.
Pre-conditions	Admin and driver logged in (two browsers).
Test Data	Admin sends “Hold CS-1001 at Surat”
Expected Result	Message saved and appears instantly in the driver's open chat.
Actual Result	Message saved and appears instantly in the driver's open chat.
Status	Pass

Test Case 9 — AI Assistant with Fallback

Field	Details
Test ID	TC009
Description	Verify the assistant answers from live data, with or without Gemini.
Pre-conditions	Shipments and open disruptions exist.
Test Data	“Give me a summary” with valid and invalid key
Expected Result	Gemini answer with valid key; offline engine answers with invalid key.
Actual Result	Gemini answer with valid key; offline engine answers with invalid key.
Status	Pass

Test Case 10 — Reports and CSV Export

Field	Details
Test ID	TC010
Description	Verify reports render and export for managers with date filter.
Pre-conditions	Manager logged in; data exists.
Test Data	/reports?from=2026-09-01&to=2026-09-30; disruptions.csv
Expected Result	Four reports shown; CSV opens in Excel; filter limits rows.
Actual Result	Four reports shown; CSV opens in Excel; filter limits rows.
Status	Pass

Test Case 11 — User Deactivation

Field	Details
Test ID	TC011
Description	Verify a deactivated user cannot log in.
Pre-conditions	Admin logged in; target user has a session.
Test Data	Admin clicks Deactivate
Expected Result	Login refused with message; open session logged out.
Actual Result	Login refused with message; open session logged out.
Status	Pass

Test Case 12 — Demo Disruption Trigger

Field	Details
Test ID	TC012
Description	Verify the demo trigger creates a HIGH disruption instantly.
Pre-conditions	Manager logged in; in-transit shipment exists.
Test Data	Click “Trigger demo disruption”
Expected Result	HIGH disruption with ranked options, map zone and live alert appear.
Actual Result	HIGH disruption with ranked options, map zone and live alert appear.
Status	Pass

Test Case 13 — Responsive Layout and Themes

Field	Details
Test ID	TC013
Description	Verify the UI works on phones and theme preference is kept.
Pre-conditions	Browser at 390 × 844 and 1440 × 900.
Test Data	Open 8 main pages; toggle dark mode
Expected Result	No sideways scrolling on phone; drawer works; theme remembered.
Actual Result	No sideways scrolling on phone; drawer works; theme remembered.
Status	Pass

6. PROPOSED ENHANCEMENT

The current version fulfils the objectives of the MCA Semester 3 project. The following enhancements have been identified as valuable future work:

- **Real GPS / telematics integration** — replace simulated shipment positions with live feeds from vehicle trackers.
- **PostgreSQL database** — migrate from SQLite for multi-server, production-scale deployment.
- **Machine-learning risk model** — train a predictive model on historical disruption data to support or replace the current weighted formula.
- **Mobile app / PWA for field operators** — an installable app with push notifications for status updates on the move.
- **Multi-tenant support** — let several independent logistics companies use isolated instances of the platform.
- **More notification channels** — SMS and WhatsApp alerts alongside in-app notifications.
- **Customs and port-authority APIs** — verified real-time congestion data.
- **Smarter AI assistant** — better prompts, conversation memory and suggested actions.
- **PDF reports and scheduled e-mails** — PDF export and a daily risk-summary e-mail.

7. CONCLUSION

ChainShield Transportation System addresses a well-defined, real-world problem: supply-chain disruptions are usually discovered only after delivery timelines are already compromised. By continuously scoring every active shipment with a transparent, weighted risk model, and by pairing every detected disruption with concrete, ranked route alternatives, the system moves logistics management from a reactive posture to a preemptive one.

The system was built from scratch as a self-contained Python (Flask) and SQLite application, containerised with Docker so that it runs identically on any server. This makes it practical for a logistics company to deploy it on its own infrastructure, connect its own weather and AI keys, and start monitoring its shipments immediately.

Group 59 translated the requirement analysis and system design in this report into a working, tested application: a scheduled risk engine, automatic disruption detection with escalation, a multi-objective route solver, a live map, role-based dashboards, real-time notifications and chat, a Gemini-powered assistant, reports with CSV export, a full admin panel and a responsive interface with light and dark themes — verified by 101 automated checks, visual checks on desktop and phone, and a Docker deployment test.

8. BIBLIOGRAPHY

Books Referred

1. Miguel Grinberg, Flask Web Development, 2nd Edition, O'Reilly Media.
2. Eric Matthes, Python Crash Course, 3rd Edition, No Starch Press.
3. Ramez Elmasri & Shamkant B. Navathe, Fundamentals of Database Systems, Pearson.
4. Ian Sommerville, Software Engineering, 10th Edition, Pearson.

Websites Referred

1. https://flask.palletsprojects.com/
2. https://docs.python.org/3/
3. https://www.sqlite.org/docs.html
4. https://docs.docker.com/
5. https://getbootstrap.com/docs/5.3/
6. https://leafletjs.com/reference.html
7. https://flask-socketio.readthedocs.io/
8. https://apscheduler.readthedocs.io/
9. https://www.chartjs.org/docs/
10. https://ai.google.dev/gemini-api/docs
11. https://openweathermap.org/api
12. https://developer.mozilla.org/
13. https://www.geeksforgeeks.org/